

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐẠI NAM



«TÊN SINH VIÊN»

MSV: «MÃ SINH VIÊN»

**XÂY DỰNG AI VISION SERVICE CHO HỆ THỐNG SMART
CAMPUS**

BÁO CÁO BÀI TẬP LỚN NGÀNH

HÀ NỘI, NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC ĐẠI NAM



«TÊN SINH VIÊN»

MSV: «MÃ SINH VIÊN», KHÓA: «KHÓA»

**XÂY DỰNG AI VISION SERVICE CHO HỆ THỐNG SMART
CAMPUS**

BÁO CÁO BÀI TẬP LỚN

NGÀNH: CÔNG NGHỆ THÔNG TIN

CHUYÊN NGÀNH: «CHUYÊN NGÀNH»

GIẢNG VIÊN HƯỚNG DẪN: ThS. Lê Thị Thùy Trang

HÀ NỘI, NĂM 2026

Nhóm em xin cam đoan báo cáo bài tập lớn với đề tài **Xây dựng AI Vision Service cho hệ thống Smart Campus** là kết quả tìm hiểu, phân tích và thực hiện của nhóm trong học phần FIT4110 – Dịch vụ kết nối và Công nghệ nền tảng.

Các nội dung trình bày trong báo cáo được tổng hợp từ mã nguồn dự án, tài liệu kỹ thuật của nhóm và các tài liệu tham khảo có ghi nguồn. Nhóm em không sao chép nội dung của nhóm khác và xin chịu trách nhiệm về tính trung thực của báo cáo.

Hà Nội, ngày ...tháng ...năm 2026

Đại diện nhóm

«TÊN SINH VIÊN»

Nhóm em xin gửi lời cảm ơn chân thành tới Trường Đại học Đại Nam, Khoa Công nghệ thông tin và giảng viên học phần ThS. Lê Thị Thùy Trang đã hướng dẫn, cung cấp yêu cầu bài toán và hỗ trợ nhóm trong quá trình thực hiện bài tập lớn.

Thông qua đề tài AI Vision Service, nhóm có cơ hội vận dụng kiến thức về dịch vụ kết nối, thiết kế REST API với FastAPI, xử lý ảnh bằng YOLOv8n, đóng gói Docker và tích hợp nhiều service trong một nền tảng Smart Campus. Đây là trải nghiệm hữu ích giúp nhóm hiểu rõ hơn cách xây dựng một service xử lý ảnh bằng AI độc lập nhưng vẫn có khả năng phối hợp với các hệ thống khác.

Do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Nhóm em kính mong nhận được góp ý của cô để tiếp tục hoàn thiện sản phẩm và nâng cao chất lượng triển khai trong các bài toán thực tế.

Hà Nội, ngày ...tháng ...năm 2026

Nhóm thực hiện

NHÓM A4

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

TỪ VIẾT TẮT	TÊN ĐẦY ĐỦ / Ý NGHĨA
API	Application Programming Interface – giao diện lập trình ứng dụng.
CNN	Convolutional Neural Network – mạng nơ-ron tích chập.
Docker	Nền tảng đóng gói và chạy ứng dụng trong container.
HTTP	HyperText Transfer Protocol – giao thức truyền tải dữ liệu trên web.
JSON	JavaScript Object Notation – định dạng trao đổi dữ liệu nhẹ.
mock AI	AI mô phỏng – kết quả giả lập khi không chạy được model thật.
OpenAPI	Đặc tả chuẩn mô tả REST API.
REST	Representational State Transfer – phong cách thiết kế API dựa trên tài nguyên.
SVI	Switch Virtual Interface – giao diện ảo trên switch.
YOLO	You Only Look Once – mô hình phát hiện đối tượng thời gian thực.

DANH MỤC CÁC BẢNG, SƠ ĐỒ, HÌNH

BẢNG:

Tên bảng	Trang
Bảng 0.2. Danh sách thành viên nhóm thực hiện	ix
Bảng 1.1. Các service trong Smart Campus Product A	2
Bảng 1.2. VisionDetectRequest – Schema đầu vào	7
Bảng 1.3. DetectionResult – Schema đầu ra	8
Bảng 2.1. GET /health – Response	19
Bảng 2.2. Logic chuyển đổi DetectionResult → CoreCameraEvent	23
Bảng 2.3. Cấu trúc thư mục dự án AI Vision Service	24
Bảng 3.1. Trạng thái hoàn thành các chức năng	25
Bảng 3.2. Postman Test Cases	30
Bảng 3.3. Minh chứng vận hành AI Vision Service	32
Bảng 3.4. Kết quả phân tích ảnh mẫu bằng YOLOv8n	32
Bảng 3.5. Tài nguyên hệ thống khi chạy AI Vision Service	33
Bảng 4.1. So sánh giao tiếp đồng bộ và bất đồng bộ	35
Bảng 4.2. Mapping giữa AI Vision và Core Business	37
Bảng 4.3. Ma trận quyết định của AI Vision	38
Bảng 4.4. Tự đánh giá chi tiết theo Rubric	41
Bảng 4.5. Phân công công việc nhóm A4	42
Bảng 4.6. So sánh phương án lưu trữ và xác thực	44
Bảng 4.7. Provider → Consumer trong Product A	49

SƠ ĐỒ:

Tên sơ đồ	Trang
Sơ đồ 1.1. Sơ đồ Service Boundary của AI Vision Service trong Product A .	10
Sơ đồ 1.2. Sơ đồ Use-case của AI Vision Service	11
Sơ đồ 1.3. Sơ đồ Deployment – Kiến trúc Docker của AI Vision Service ...	12
Sơ đồ 2.1. Sequence Diagram – Luồng xử lý một detection request	15
Sơ đồ 4.1. Luồng xử lý End-to-End trong Smart Campus Product A.....	40

HÌNH:

Tên hình	Trang
Hình 1.1. Kiến trúc tổng quan của YOLOv8	3
Hình 2.1. Kiến trúc phân tầng của AI Vision Service	13
Hình 4.1. Swagger UI – Tài liệu API tự động của AI Vision Service	51
Hình 4.2. Postman – Kết quả kiểm thử tích hợp	51
Hình 4.3. Docker – Container AI Vision Service đang hoạt động	52

MỤC LỤC

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT	ii
DANH MỤC CÁC BẢNG, SƠ ĐỒ, HÌNH	iii
LỜI MỞ ĐẦU	viii
CHƯƠNG 1. TỔNG QUAN BÀI TOÁN VÀ PHÂN TÍCH SERVICE BOUNDARY . .	1
1.1. Giới thiệu Smart Campus Operations Platform	1
1.1.1. Bối cảnh và động lực	1
1.1.2. Kiến trúc Microservices và Event-Driven	1
1.1.3. Các service trong Product A	2
1.2. Cơ sở lý thuyết và công nghệ	2
1.2.1. YOLO – Mô hình phát hiện đối tượng thời gian thực	2
1.2.2. FastAPI – Web framework hiện đại cho Python	3
1.2.3. Docker và Containerization	4
1.3. Service nhóm phụ trách – AI Vision Service	4
1.3.1. Vai trò trong hệ thống	4
1.3.2. Mục tiêu cụ thể	5
1.4. Phạm vi chức năng	5
1.4.1. Các chức năng nằm trong phạm vi (In-Scope)	5
1.4.2. Giới hạn của hệ thống	6
1.5. Input và Output của service	7
1.5.1. Dữ liệu đầu vào (Input)	7
1.5.2. Dữ liệu đầu ra (Output)	7
1.6. Upstream và Downstream	9
1.6.1. Quan hệ Provider – Consumer	9
1.6.2. Dịch vụ Upstream	9
1.6.3. Dịch vụ Downstream	9
1.7. Sơ đồ Service Boundary	9
1.8. Sơ đồ Use-case	10
1.9. Sơ đồ Deployment (Triển khai)	11
1.10. Kết luận chương	12
CHƯƠNG 2. THIẾT KẾ HỆ THỐNG VÀ API CONTRACT	13
2.1. Kiến trúc tổng quan	13
2.1.1. Tầng API (Presentation Layer)	13
2.1.2. Tầng Nghiệp vụ (Business Logic Layer)	13

2.1.3.	Tầng Tích hợp (Integration Layer)	14
2.1.4.	Tầng Hạ tầng (Infrastructure Layer)	14
2.2.	Luồng xử lý chi tiết – Sequence Diagram	14
2.2.1.	Diễn giải từng bước	15
2.3.	Thiết kế Data Models	16
2.3.1.	VisionDetectRequest – Schema đầu vào	16
2.3.2.	DetectionResult – Schema đầu ra chính	16
2.3.3.	Discriminated Union – PersonFinding và ObjectFinding	17
2.3.4.	CoreCameraEvent – Adapter cho Core Business	18
2.3.5.	ProblemDetails – RFC 9457 Error Model	18
2.4.	API Endpoints chi tiết	18
2.4.1.	GET / – Dashboard	18
2.4.2.	GET /health	19
2.4.3.	GET /model	19
2.4.4.	POST /api/v1/vision/detect – Endpoint chính	20
2.4.5.	GET /api/v1/vision/detections/{detection_id}	21
2.4.6.	POST /api/v1/events/camera	21
2.4.7.	POST /analyze và POST /analyze/mock	21
2.5.	Mã nguồn tiêu biểu và giải thích	21
2.5.1.	Luồng xử lý chính – <code>_run_detection()</code>	21
2.5.2.	Adapter – <code>_map_to_core_camera_event()</code>	22
2.6.	Cấu trúc Repository	24
2.7.	Kết luận chương	24
CHƯƠNG 3.	CÀI ĐẶT, KIỂM THỬ VÀ ĐÓNG GÓI DOCKER	25
3.1.	Tổng quan các chức năng đã cài đặt	25
3.2.	Phân tích chi tiết các module	25
3.2.1.	<code>app/vision.py</code> – AI Engine (146 dòng)	25
3.2.2.	<code>app/main.py</code> – API Server (940 dòng)	28
3.2.3.	<code>test_api.py</code> – Smoke Test Automation (65 dòng)	29
3.3.	Kiểm thử tích hợp – Postman Collection	30
3.4.	Đóng gói Docker	30
3.4.1.	<code>Dockerfile</code>	30
3.4.2.	<code>docker-compose.yml</code>	31
3.4.3.	Kiểm tra container đang chạy	31
3.5.	Minh chứng vận hành	32
3.6.	Đánh giá hiệu năng mô hình YOLO	32
3.6.1.	Phân tích kết quả	32

3.6.2. Tài nguyên tiêu thụ	33
3.7. Kết luận chương	33
CHƯƠNG 4. TÍCH HỢP VÀ KẾT NỐI LIÊN SERVICE	35
4.1. Tổng quan về tích hợp trong Product A	35
4.2. Mô hình tích hợp	35
4.2.1. Phân biệt đồng bộ và bất đồng bộ	35
4.3. Kết nối A2 (Camera Stream) → A4 (AI Vision)	36
4.3.1. Mô tả luồng kết nối	36
4.3.2. Hợp đồng API	36
4.3.3. Phân tích độ tin cậy	36
4.4. Kết nối A4 (AI Vision) → A6 (Core Business)	37
4.4.1. Vai trò của Core Business	37
4.4.2. Adapter Pattern	37
4.4.3. Logic tự động ra quyết định	37
4.5. Kết nối A4 (AI Vision) → A5 (Analytics)	38
4.5.1. Vai trò của Analytics	38
4.5.2. Hợp đồng dữ liệu	38
4.6. Luồng xử lý End-to-End	39
4.7. Tự đánh giá theo Rubric học phần	40
4.8. Phân công công việc	42
4.9. Hạn chế và hướng phát triển	42
4.9.1. Các điểm hạn chế hiện tại	42
4.9.2. Hướng phát triển trong tương lai	42
4.10. Bảo mật và Lưu trữ dữ liệu	43
4.10.1. Hiện trạng	43
4.10.2. Giải pháp đề xuất	43
4.10.3. So sánh các phương án	44
4.11. Kết luận chương	44
KẾT LUẬN	45
PHỤ LỤC	48
TÀI LIỆU THAM KHẢO	53

LỜI MỞ ĐẦU

Bài toán

Trong hệ thống Smart Campus, camera IP được lắp đặt tại các khu vực trọng yếu như cổng chính, hành lang, bãi xe. Tuy nhiên, camera chỉ ghi hình – nó không tự hiểu được **đang có ai ở đó, có chuyển động bất thường hay không**. Cần một service AI có khả năng nhận ảnh từ camera, phân tích và trả về kết quả phát hiện để các service khác ra quyết định.

Cụ thể, khi Camera Stream Service (A2) phát hiện chuyển động, nó cần gửi frame ảnh đến một service phân tích. Service đó phải trả lời được: **Trong ảnh có đối tượng gì? Người hay xe? Độ tin cậy bao nhiêu? Có đáng báo động không?**

Giải pháp

Nhóm A4 xây dựng AI Vision Service – service phân tích ảnh bằng YOLOv8n. Service nhận frame_url từ A2, tải ảnh về, chạy mô hình YOLOv8n để phát hiện đối tượng, và trả về kết quả JSON gồm: loại đối tượng, độ tin cậy, mức độ rủi ro. Nếu không chạy được YOLO (thiếu GPU, thiếu thư viện), service tự fallback sang mock AI để vẫn demo được.

Kết quả phân tích được gửi đến Core Business (A6) để ra quyết định cảnh báo, và Analytics (A5) để tổng hợp thống kê.

Mục tiêu

1. Nhận frame_url từ Camera Stream, tải ảnh và phân tích bằng YOLOv8n.
2. Phát hiện đối tượng (người, xe, động vật...) và phân loại mức độ rủi ro (low/medium/high).
3. Cung cấp kết quả cho Core Business qua adapter endpoint, kèm đề xuất quyết định (no_alert / alert_required).
4. Cung cấp detection log cho Analytics để tổng hợp thống kê.
5. Đóng gói Docker, có thể chạy từ repo sạch với một lệnh duy nhất.

Phạm vi

- **Trong phạm vi:** Xử lý ảnh tĩnh (JPEG/PNG) qua URL; YOLOv8n inference + mock fallback; REST API 7 endpoint; Docker + Compose; Postman test; OpenAPI 3.1.0.
- **Ngoài phạm vi:** Huấn luyện mô hình mới; xử lý video stream; nhận diện khuôn mặt; lưu trữ ảnh lâu dài; authentication.

Công nghệ sử dụng: Python 3.11, FastAPI, YOLOv8n (Ultralytics), OpenCV, httpx, Docker.

Bảng 0.2. Danh sách thành viên nhóm thực hiện

STT	Họ và tên	Mã sinh viên
1	«THÀNH VIÊN 1»	«MSSV 1»
2	«THÀNH VIÊN 2»	«MSSV 2»
3	«THÀNH VIÊN 3»	«MSSV 3»

Nguồn: Nhóm thực hiện

Chương 1

TỔNG QUAN BÀI TOÁN VÀ PHÂN TÍCH SERVICE BOUNDARY

1.1 Giới thiệu Smart Campus Operations Platform

1.1.1 Bối cảnh và động lực

Trong kỷ nguyên chuyển đổi số, các trường đại học trên thế giới đang hướng tới mô hình **Smart Campus** – khuôn viên thông minh, nơi mọi hoạt động từ giám sát an ninh, quản lý ra vào, theo dõi môi trường đến thông báo sự cố đều được số hóa và tự động hóa. Một Smart Campus không chỉ đơn thuần là lắp đặt camera hay cảm biến, mà còn đòi hỏi một **nền tảng phần mềm** có khả năng tiếp nhận dữ liệu từ nhiều nguồn, xử lý thông minh và phản ứng kịp thời.

Smart Campus Operations Platform được thiết kế như một hệ thống mô phỏng vận hành thông minh trong khuôn viên trường đại học. Nền tảng này tích hợp 7 service độc lập, mỗi service đảm nhận một vai trò chuyên biệt, phối hợp với nhau tạo thành một hệ thống hoàn chỉnh có khả năng trả lời các câu hỏi vận hành như:

- Có người lạ xuất hiện trong khu vực cấm không?
- Nhiệt độ, độ ẩm, chuyển động có vượt ngưỡng an toàn không?
- Có lượt ra/vào bất thường ngoài giờ không?
- Khi có sự kiện bất thường, hệ thống có gửi thông báo kịp thời không?
- Có thống kê được số lượt truy cập, số cảnh báo, trạng thái cảm biến không?

1.1.2 Kiến trúc Microservices và Event-Driven

Hệ thống được thiết kế theo **kiến trúc Microservices** kết hợp với **Kiến trúc hướng sự kiện** (Event-Driven Architecture – EDA). Đây là hai mẫu kiến trúc hiện đại, được các hệ thống lớn như Netflix, Amazon, Uber áp dụng.

Trong kiến trúc Microservices, thay vì xây dựng một khối nguyên khối (monolithic), hệ thống được chia thành nhiều service nhỏ, độc lập. Mỗi service:

- Có **ranh giới rõ ràng** (service boundary) – biết mình nhận gì, trả gì.

- Có **cơ sở dữ liệu riêng** hoặc cơ chế lưu trữ độc lập.
- Có thể được **phát triển, kiểm thử và triển khai độc lập**.
- Giao tiếp với nhau qua **REST API** (đồng bộ) hoặc **message queue** (bất đồng bộ).
- Có thể sử dụng **công nghệ khác nhau** – Python, Node.js, Go... miễn là tuân thủ hợp đồng API.

Kiến trúc hướng sự kiện bổ sung khả năng giao tiếp bất đồng bộ: khi một service phát hiện sự kiện (ví dụ: phát hiện chuyển động từ camera), nó publish sự kiện đó lên message broker. Các service khác subscribe để nhận và xử lý. Điều này giúp hệ thống **lỏng lẻo về mặt phụ thuộc** (loose coupling) – mỗi service không cần biết chi tiết implementation của service khác.

1.1.3 Các service trong Product A

Toàn bộ Product A bao gồm 7 service:

Bảng 1.1. Các service trong Smart Campus Product A

Mã nhóm	Tên service	Vai trò	Độ phức tạp
A1	IoT Ingestion	Tiếp nhận dữ liệu cảm biến	Trung bình
A2	Camera Stream	Quản lý luồng camera	Trung bình–Khó
A3	Access Gate	Kiểm soát ra/vào RFID	Trung bình
A4	AI Vision	Phân tích ảnh bằng AI	Khó
A5	Analytics	Tổng hợp thống kê	Khó
A6	Core Business	Xử lý nghiệp vụ trung tâm	Khó nhất
A7	Notification	Gửi thông báo đa kênh	Trung bình

Nguồn: Tài liệu học phần FIT4110

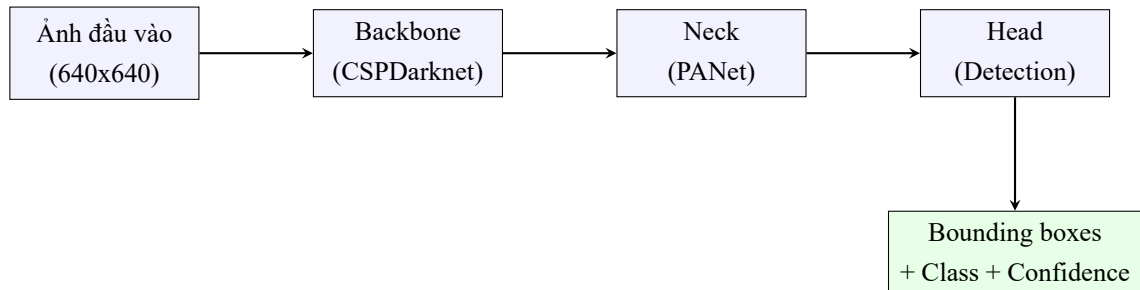
1.2 Cơ sở lý thuyết và công nghệ

Trước khi đi vào thiết kế chi tiết, phần này trình bày các công nghệ nền tảng được sử dụng trong AI Vision Service.

1.2.1 YOLO – Mô hình phát hiện đối tượng thời gian thực

YOLO (You Only Look Once) là một trong những mô hình phát hiện đối tượng (object detection) phổ biến nhất hiện nay, được phát triển bởi Joseph Redmon và sau đó được

Ultralytics tiếp tục phát triển. Khác với các phương pháp truyền thống sử dụng sliding window hoặc region proposal, YOLO áp dụng một cách tiếp cận khác biệt: **nhìn toàn bộ ảnh đúng một lần** và dự đoán đồng thời bounding box lẫn class probability.



Hình 1.1. Kiến trúc tổng quan của YOLOv8

Nguồn: *Ultralytics YOLOv8 Documentation*

Các đặc điểm nổi bật của YOLO:

- **Tốc độ cao:** YOLOv8n (nano) có thể xử lý hàng trăm frame/giây trên GPU, phù hợp với bài toán thời gian thực.
- **Độ chính xác tốt:** YOLOv8 đạt mAP (mean Average Precision) cao trên các benchmark như COCO.
- **Dễ triển khai:** Ultralytics cung cấp API Python đơn giản, chỉ cần vài dòng code để chạy inference.
- **Nhiều phiên bản:** Từ YOLOv8n (nano – nhẹ nhất) đến YOLOv8x (extra large), phù hợp với nhiều điều kiện phần cứng.

Trong AI Vision Service, nhóm sử dụng **YOLOv8n** – phiên bản nhẹ nhất, phù hợp để triển khai trong môi trường học tập và demo, đồng thời vẫn đảm bảo độ chính xác ở mức chấp nhận được.

1.2.2 FastAPI – Web framework hiện đại cho Python

FastAPI là web framework Python được xây dựng trên nền tảng Starlette và Pydantic, nổi bật với:

- **Hiệu năng cao:** FastAPI là một trong những Python framework nhanh nhất, ngang ngửa với Node.js và Go.
- **Type hints:** Tận dụng Python type hints để tự động validate request/response, sinh tài liệu API.

- **OpenAPI tự động:** Swagger UI và ReDoc được sinh tự động từ code, không cần viết riêng.
- **Async support:** Hỗ trợ async/await, phù hợp với các tác vụ I/O như tải ảnh từ URL.

1.2.3 Docker và Containerization

Docker là nền tảng đóng gói ứng dụng vào container – một đơn vị phần mềm chuẩn hóa, chứa mọi thứ cần thiết để chạy ứng dụng: code, runtime, thư viện, biến môi trường. Docker giải quyết bài toán “chạy được trên máy tôi nhưng không chạy được trên máy bạn” – một vấn đề kinh điển trong phát triển phần mềm.

Lợi ích của Docker trong bài tập lớn:

- **Nhất quán môi trường:** Mọi thành viên và giảng viên đều chạy service trong môi trường giống hệt nhau.
- **Dễ triển khai:** Chỉ cần một lệnh `docker compose up` để khởi động toàn bộ service.
- **Cô lập:** Mỗi service chạy trong container riêng, không xung đột thư viện hay port.
- **Chuẩn công nghiệp:** Docker là kỹ năng bắt buộc trong các vị trí DevOps và Backend Developer.

1.3 Service nhóm phụ trách – AI Vision Service

Nhóm A4 phụ trách xây dựng và vận hành **AI Vision Service** – service phân tích hình ảnh bằng trí tuệ nhân tạo. Đây là một trong những service có độ phức tạp cao trong hệ thống Smart Campus, đòi hỏi sự kết hợp giữa kiến thức về REST API, xử lý ảnh và machine learning.

1.3.1 Vai trò trong hệ thống

AI Vision Service đóng vai trò là **AI Engine** của toàn bộ nền tảng. Trong kiến trúc tổng thể, AI Vision nằm ở giữa chuỗi xử lý camera:

- **Phía trước (upstream):** Camera Stream Service (A2) – nơi cung cấp ảnh đầu vào.
- **Phía sau (downstream):** Core Business (A6) và Analytics (A5) – nơi tiêu thụ kết quả phân tích.

Nếu ví hệ thống Smart Campus như cơ thể con người, thì Camera Stream là “mắt”, AI Vision là “não thị giác” – nơi phân tích những gì mắt nhìn thấy, và Core Business là “não trung tâm” – nơi ra quyết định dựa trên thông tin đã phân tích.

1.3.2 Mục tiêu cụ thể

1. **Nhận ảnh từ Camera Stream:** Service cung cấp REST API endpoint để Camera Stream gửi frame_url khi phát hiện chuyển động.
2. **Tải và giải mã ảnh:** Sử dụng httpx để tải ảnh từ URL và OpenCV để giải mã.
3. **Chạy suy luận AI:** Sử dụng YOLOv8n để phát hiện đối tượng trong ảnh. Kết quả bao gồm: loại đối tượng (person, vehicle, bicycle...), vị trí (bounding box), độ tin cậy (confidence score).
4. **Fallback thông minh:** Nếu YOLO không khả dụng (môi trường không có GPU, thiếu thư viện), service tự động chuyển sang chế độ mock AI – sinh kết quả giả lập có cấu trúc giống hệt kết quả thật. Điều này đảm bảo **service luôn sẵn sàng cho demo và kiểm thử**.
5. **Chuẩn hóa kết quả:** Mọi kết quả (dù từ YOLO thật hay mock) đều được chuẩn hóa về cùng một JSON schema, đảm bảo consumer (Core Business, Analytics) không cần biết service đang chạy ở chế độ nào.
6. **Cung cấp adapter cho Core Business:** Endpoint /api/v1/events/camera chuyển đổi kết quả AI Vision thành schema CoreCameraEvent mà Core Business yêu cầu.
7. **Cung cấp detection log cho Analytics:** Dữ liệu phát hiện được cấu trúc để Analytics có thể tổng hợp thống kê.

1.4 Phạm vi chức năng

1.4.1 Các chức năng nằm trong phạm vi (In-Scope)

- **REST API đầy đủ:** 7 endpoint bao gồm health check, model status, detection, lookup, adapter và mock demo.
- **Xử lý ảnh:** Tải ảnh từ URL, giải mã bằng OpenCV, chạy YOLOv8n inference.
- **Fallback mock AI:** Tự động phát hiện và chuyển sang mock khi model thật không khả dụng.

- **Phân loại rủi ro:** Hệ thống 3 mức – low, medium, high – dựa trên loại đối tượng và confidence score.
- **Lưu trữ kết quả:** In-memory dictionary cho phép tra cứu lại kết quả theo detection_id.
- **Dashboard:** Giao diện web dark-theme hiển thị trạng thái service và form test API.
- **Adapter cho Core Business:** Tự động chuyển đổi DetectionResult thành CoreCameraEvent.
- **OpenAPI 3.1.0:** File đặc tả API đầy đủ, có thể xem qua Swagger UI.
- **Docker:** Dockerfile + docker-compose.yml, chạy được từ repo sạch.
- **Kiểm thử:** Postman Collection với 7 test case + smoke test Python script.
- **Minh chứng vận hành:** Log, screenshot, health check, model status.

1.4.2 Giới hạn của hệ thống

- **Huấn luyện mô hình mới:** Service chỉ sử dụng mô hình YOLOv8n đã được huấn luyện sẵn trên COCO dataset. Việc fine-tune hoặc huấn luyện từ đầu đòi hỏi tài nguyên GPU và thời gian không phù hợp với phạm vi học phần.
- **Xử lý video stream:** Service xử lý từng frame ảnh tĩnh, không xử lý video stream liên tục. Điều này giúp đơn giản hóa thiết kế và phù hợp với mô hình request-response của REST API.
- **Nhận diện khuôn mặt:** YOLOv8n có thể phát hiện “person” nhưng không phân biệt được danh tính cụ thể. Nhận diện khuôn mặt (face recognition) là bài toán riêng, đòi hỏi mô hình và pipeline khác.
- **Lưu trữ ảnh lâu dài:** Ảnh chỉ được tải về để phân tích rồi giải phóng. Việc lưu trữ ảnh thuộc về Camera Stream Service.
- **Giao diện người dùng phức tạp:** Dashboard HTML chỉ phục vụ mục đích demo và kiểm tra nhanh, không phải production UI.
- **Authentication/Authorization:** Trong phạm vi học phần, các endpoint không yêu cầu xác thực.

1.5 Input và Output của service

1.5.1 Dữ liệu đầu vào (Input)

AI Vision Service nhận request từ Camera Stream Service với cấu trúc JSON:

Bảng 1.2. VisionDetectRequest – Schema đầu vào

Trường	Kiểu dữ liệu	Bắt buộc	Mô tả
camera_id	string	Có	Mã định danh camera (vd: cam-gate-01)
frame_url	string (URL)	Có	URL của ảnh cần phân tích. Hỗ trợ alias “image_”
timestamp	datetime (ISO 8601)	Có	Thời điểm camera ghi nhận frame

Nguồn: Nhóm thực hiện

Ví dụ request thực tế:

POST /api/v1/vision/detect

Content-Type: application/json

```
{
  "camera_id": "cam-gate-01",
  "frame_url": "http://26.231.100.34:8080/frame.jpg",
  "timestamp": "2026-06-17T01:18:39"
}
```

1.5.2 Dữ liệu đầu ra (Output)

Kết quả phân tích được trả về dưới dạng JSON:

Bảng 1.3. DetectionResult – Schema đầu ra

Trường	Kiểu	Ý nghĩa
detection_id	string	Mã duy nhất (vd: det-a1b2c3d4e5f6)
camera_id	string	Mã camera từ request
frame_url	string	URL ảnh đã phân tích
timestamp	datetime	Thời điểm phân tích (UTC)
anomaly_detected	boolean	Có phát hiện bất thường?
confidence_score	float (0–1)	Độ tin cậy (chỉ có khi anomaly_detected = true)
finding	object	Chi tiết đối tượng (PersonFinding/ObjectFinding)
model_mode	string	“yolo” hoặc “mock-ai”
notes	string (optional)	Ghi chú bổ sung

Nguồn: Nhóm thực hiện

Ví dụ response:

```
{
  "detection_id": "det-0ed98880e029",
  "camera_id": "cam-gate-01",
  "frame_url": "http://26.231.100.34:8080/frame.jpg",
  "timestamp": "2026-06-17T01:18:39",
  "anomaly_detected": true,
  "confidence_score": 0.87,
  "finding": {
    "finding_type": "object",
    "object_name": "bus",
    "risk_level": "medium",
    "description": "Detected object: bus"
  },
  "model_mode": "yolo",
  "notes": "YOLO model: yolov8n.pt"
}
```

1.6 Upstream và Downstream

1.6.1 Quan hệ Provider – Consumer

Trong kiến trúc Microservices, mối quan hệ giữa các service được mô hình hóa theo mẫu Provider–Consumer:

- **Provider:** Service cung cấp dữ liệu hoặc chức năng cho service khác.
- **Consumer:** Service sử dụng dữ liệu hoặc chức năng từ provider.

AI Vision Service vừa là **Consumer** (của A2 – Camera Stream) vừa là **Provider** (cho A6 – Core Business và A5 – Analytics).

1.6.2 Dịch vụ Upstream

- **A2 – Camera Stream Service:** Là bên gọi chính vào AI Vision. Camera Stream liên tục nhận frame từ camera IP. Khi phát hiện chuyển động (motion detection), nó tạo frame_url và gọi POST /api/v1/vision/detect. Đây là mối quan hệ **đồng bộ** (synchronous): Camera Stream gửi request và chờ response.

1.6.3 Dịch vụ Downstream

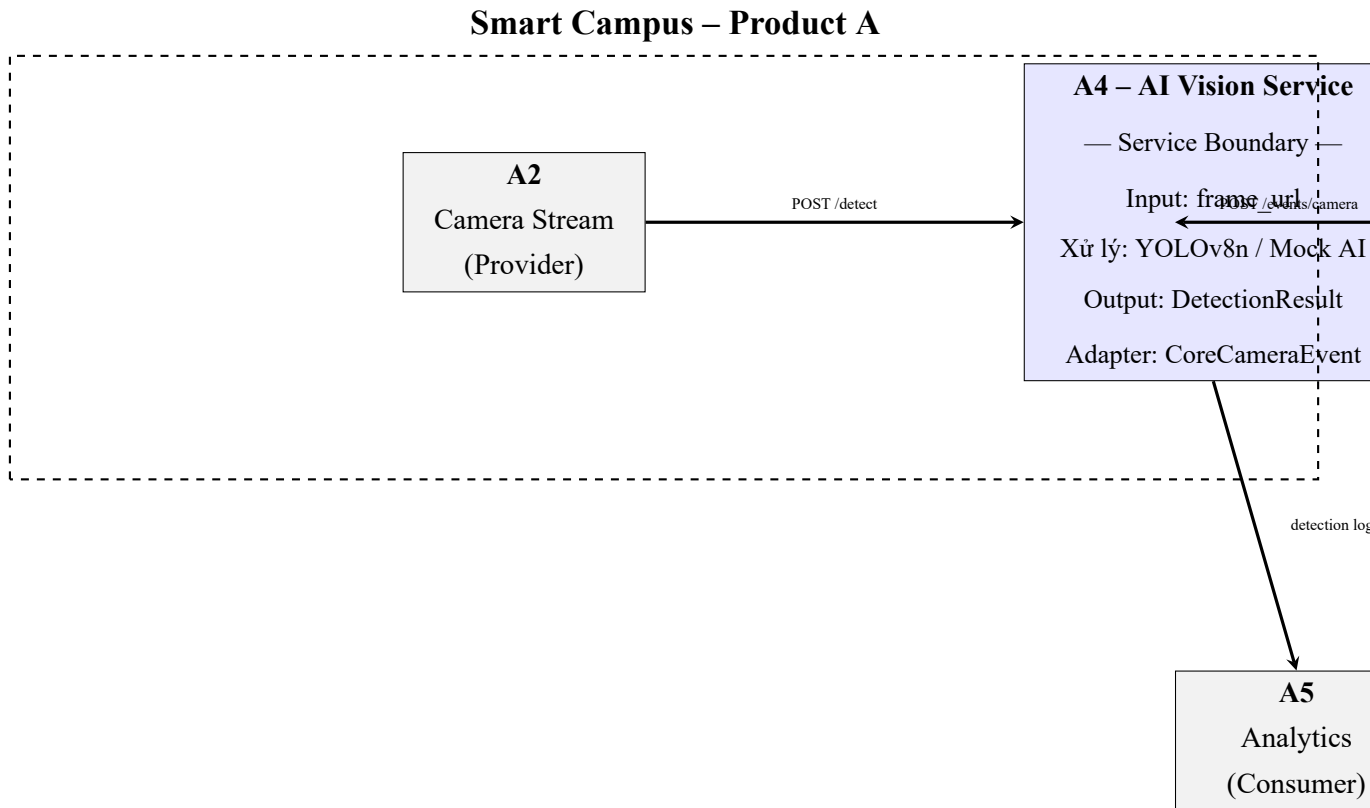
- **A6 – Core Business Service:** Nhận kết quả phát hiện từ AI Vision qua adapter endpoint POST /api/v1/events/camera. Core Business sử dụng kết quả này để áp dụng **rule nghiệp vụ**: nếu risk_level = “warning” hoặc “critical” và core_decision = “alert_required”, Core Business sẽ tạo alert và gửi sang Notification Service (A7).
- **A5 – Analytics Service:** Nhận **detection log** từ AI Vision để tổng hợp thống kê. Analytics có thể phân tích theo nhiều chiều: số lần phát hiện theo camera, phân bố loại đối tượng, tỉ lệ cảnh báo theo giờ, xu hướng theo thời gian, v.v.

1.7 Sơ đồ Service Boundary

Service Boundary là một khái niệm quan trọng trong thiết kế Microservices. Nó xác định **ranh giới trách nhiệm** của service: service làm gì, nhận gì, trả gì, phụ thuộc vào ai và ai phụ thuộc vào mình. Một service boundary rõ ràng giúp:

- Giảm coupling giữa các service.

- Mỗi nhóm có thể phát triển độc lập.
- Dễ dàng thay đổi implementation bên trong mà không ảnh hưởng đến service khác.

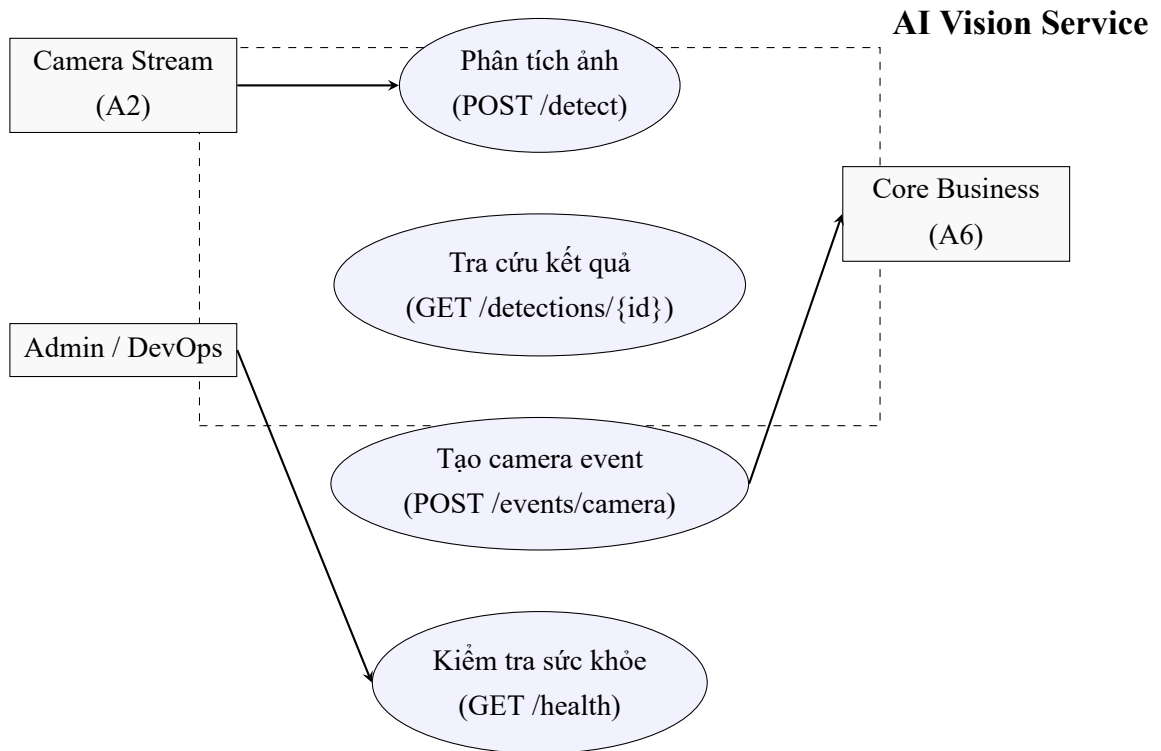


Sơ đồ 1.1. Sơ đồ Service Boundary của AI Vision Service trong Product A

Nguồn: Nhóm thực hiện

1.8 Sơ đồ Use-case

Sơ đồ dưới đây mô tả các tác nhân (actor) tương tác với AI Vision Service và các chức năng chính mà service cung cấp:

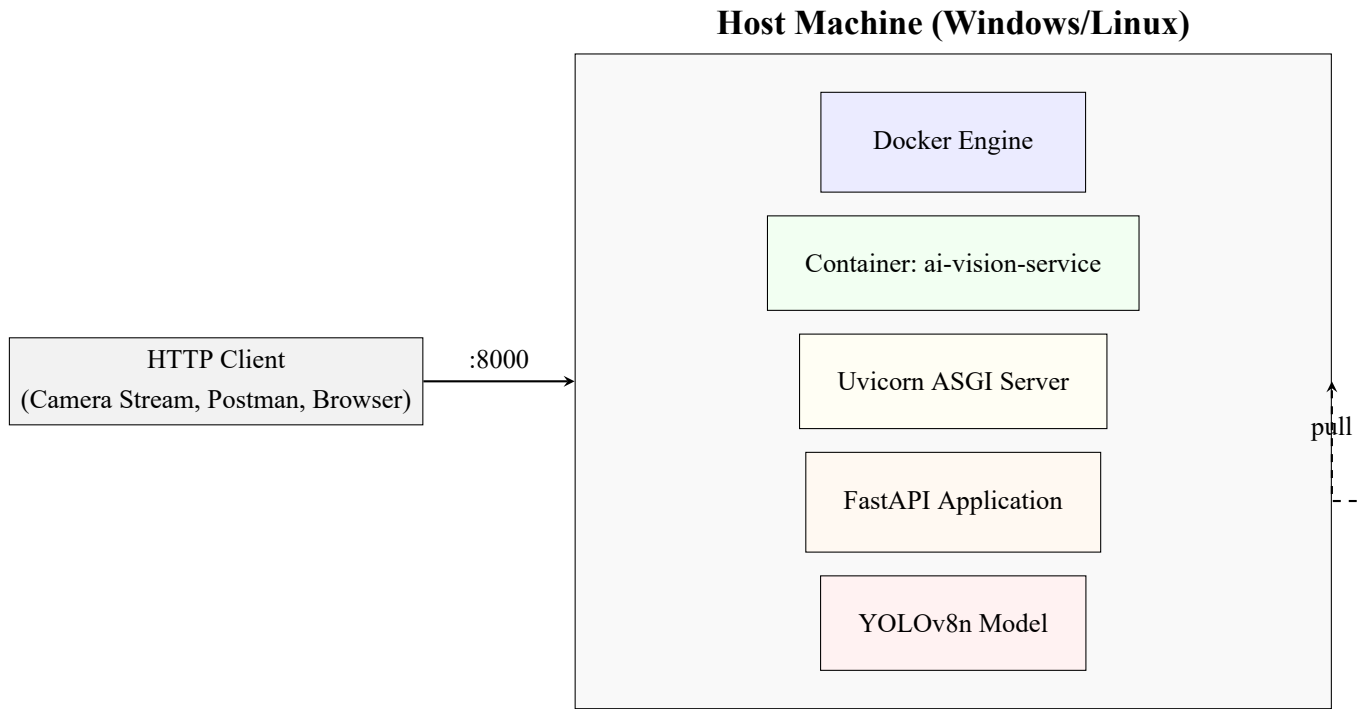


Sơ đồ 1.2. Sơ đồ Use-case của AI Vision Service

Nguồn: Nhóm thực hiện

1.9 Sơ đồ Deployment (Triển khai)

Sơ đồ dưới đây mô tả kiến trúc triển khai AI Vision Service trên Docker:



Sơ đồ 1.3. Sơ đồ Deployment – Kiến trúc Docker của AI Vision Service

Nguồn: Nhóm thực hiện

1.10 Kết luận chương

Chương này đã trình bày:

- Bối cảnh và kiến trúc tổng thể của Smart Campus Operations Platform với 7 service trong Product A.
- Cơ sở lý thuyết về các công nghệ nền tảng: YOLOv8, FastAPI, Docker và kiến trúc Microservices.
- Vai trò của AI Vision Service như “não thị giác” của hệ thống, với 7 mục tiêu cụ thể.
- Phạm vi chức năng rõ ràng, phân biệt giữa in-scope (11 chức năng) và out-of-scope (6 giới hạn).
- Phân tích chi tiết Input/Output với schema và ví dụ thực tế.
- Mối quan hệ Provider–Consumer: AI Vision là Consumer của A2, Provider cho A6 và A5.
- Sơ đồ Service Boundary thể hiện trực quan ranh giới và kết nối.

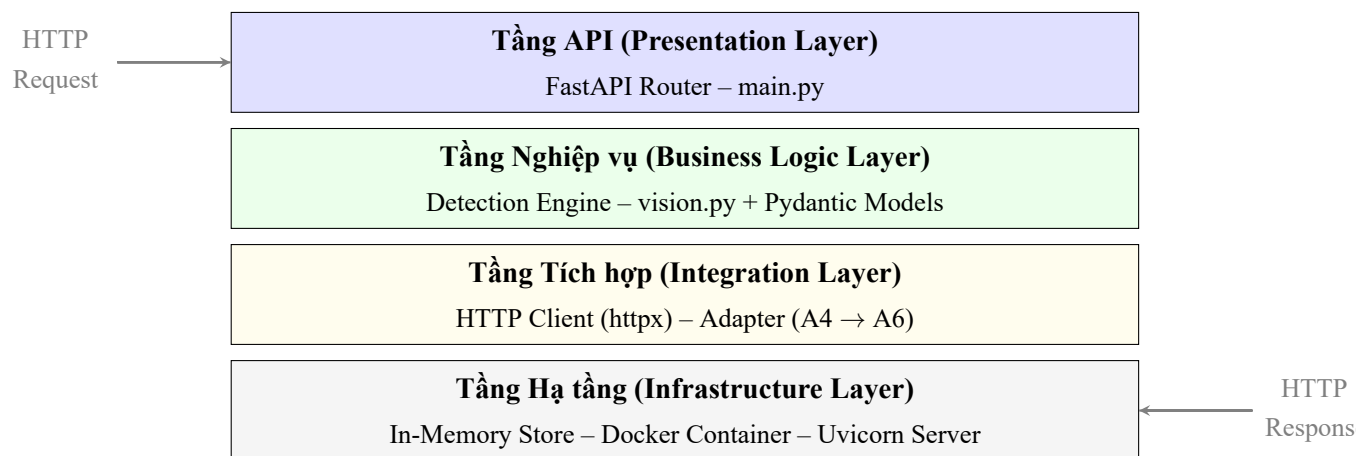
Chương tiếp theo sẽ đi sâu vào thiết kế kỹ thuật: kiến trúc bên trong service, thiết kế API, data models và luồng xử lý.

Chương 2

THIẾT KẾ HỆ THỐNG VÀ API CONTRACT

2.1 Kiến trúc tổng quan

AI Vision Service được thiết kế theo mô hình **layered architecture** với 4 tầng chính, mỗi tầng đảm nhận một trách nhiệm riêng biệt. Thiết kế này tuân theo nguyên lý **Separation of Concerns** – mỗi thành phần chỉ lo một việc, giúp code dễ đọc, dễ test và dễ bảo trì.



Hình 2.1. Kiến trúc phân tầng của AI Vision Service

Nguồn: Nhóm thực hiện

2.1.1 Tầng API (Presentation Layer)

Đây là tầng tiếp nhận HTTP request từ bên ngoài. FastAPI tự động:

- Parse và validate request body dựa trên Pydantic schema.
- Trả về HTTP 422 Unprocessable Entity nếu dữ liệu không hợp lệ (kèm thông báo lỗi chi tiết).
- Sinh OpenAPI document tự động tại /docs và /redoc.
- Xử lý CORS headers cho phép frontend gọi API.

2.1.2 Tầng Nghiệp vụ (Business Logic Layer)

Đây là “trái tim” của service, chứa toàn bộ logic xử lý ảnh:

- **analyze_image()**: Hàm entry point, quyết định dùng YOLO hay mock.
- **_run_yolo()**: Chạy inference thực tế, trích xuất bounding boxes, class names, confidence scores.
- **_mock_result()**: Sinh kết quả giả lập có cấu trúc giống hệt YOLO.
- **_risk_from_confidence()**: Phân loại rủi ro dựa trên loại đối tượng và confidence.

2.1.3 Tầng Tích hợp (*Integration Layer*)

Xử lý giao tiếp với thế giới bên ngoài:

- **_fetch_image_bytes()**: Tải ảnh từ frame_url bằng httpx với timeout và retry.
- **_map_to_core_camera_event()**: Adapter chuyển đổi DetectionResult → CoreCameraEvent

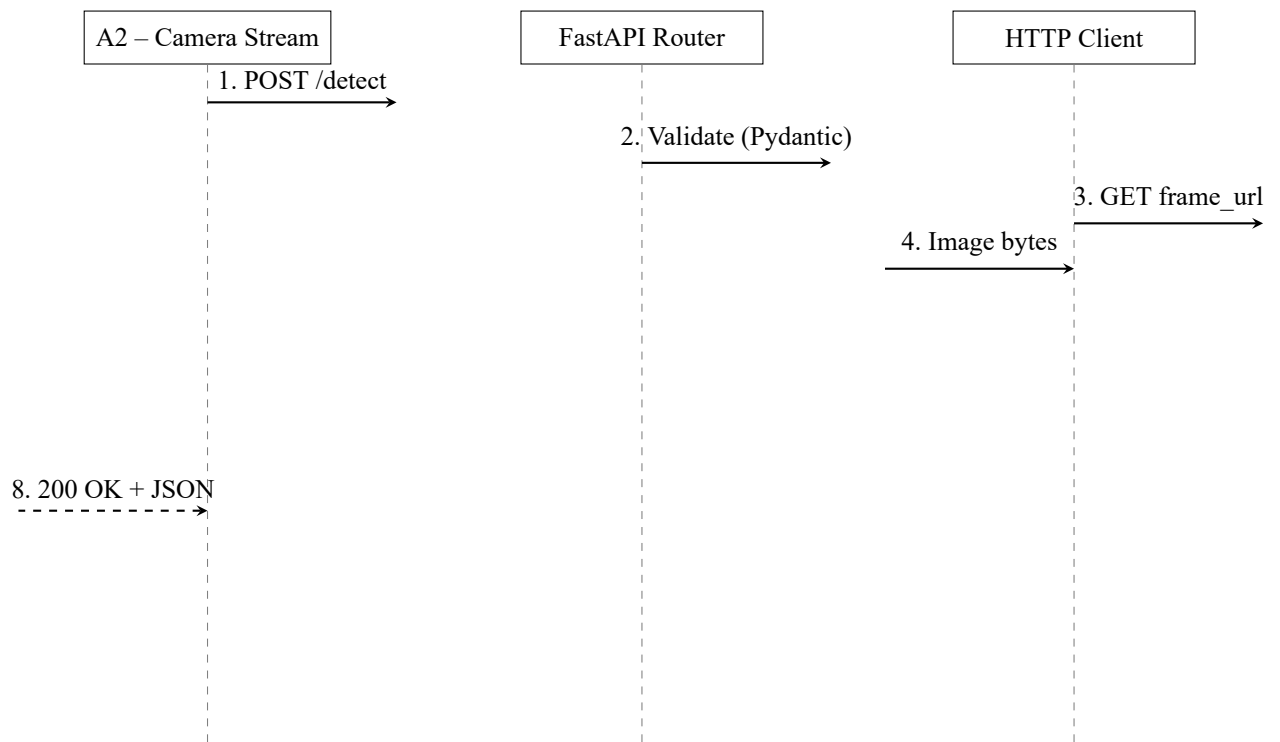
2.1.4 Tầng Hạ tầng (*Infrastructure Layer*)

Cung cấp nền tảng vận hành:

- **DETECTION_STORE**: In-memory Python dict, lưu kết quả theo detection_id.
- **Docker Container**: Môi trường chạy chuẩn hóa.
- **Uvicorn**: ASGI server, chạy FastAPI app với hiệu năng cao.

2.2 Luồng xử lý chi tiết – Sequence Diagram

Khi Camera Stream gửi request đến AI Vision, luồng xử lý diễn ra qua 8 bước. Sơ đồ sequence dưới đây mô tả tương tác giữa các thành phần theo thời gian:



Sơ đồ 2.1. Sequence Diagram – Luồng xử lý một detection request

Nguồn: Nhóm thực hiện

2.2.1 Diễn giải từng bước

1. **Camera Stream gửi request:** A2 gọi POST `/api/v1/vision/detect` với JSON body chứa `camera_id`, `frame_url`, `timestamp`.
2. **Validate dữ liệu:** FastAPI tự động validate request body qua Pydantic `VisionDetectRequest`. Nếu thiếu trường hoặc sai kiểu, trả về 422 ngay lập tức.
3. **Tải ảnh:** Hàm `_fetch_image_bytes()` sử dụng `httpx.AsyncClient` để tải ảnh từ `frame_url` với timeout 15 giây và tự động follow redirect.
4. **Nhận bytes ảnh:** Ảnh được tải về dưới dạng bytes. Service log kích thước ảnh để debug.
5. **Chạy inference:** `analyze_image()` quyết định dùng YOLO hay mock. Nếu YOLO, ảnh được giải mã bằng OpenCV rồi đưa vào `model.predict()`.
6. **Lưu kết quả:** `DetectionResult` được lưu vào `DETECTION_STORE` với key = `detection_id`.
7. **Tạo response:** Kết quả được serialize thành JSON và trả về.
8. **Trả về cho Camera Stream:** HTTP 200 OK kèm JSON body.

2.3 Thiết kế Data Models

AI Vision Service sử dụng Pydantic để định nghĩa toàn bộ schema. Pydantic đảm bảo type safety tại runtime: nếu dữ liệu không khớp schema, request bị từ chối với thông báo lỗi rõ ràng.

2.3.1 *VisionDetectRequest – Schema đầu vào*

```
class VisionDetectRequest(BaseModel):
    camera_id: str
    frame_url: HttpUrl = Field(
        validation_alias=AliasChoices("frame_url", "image_url"),
        serialization_alias="frame_url"
    )
    timestamp: datetime
```

Điều đáng chú ý:

- **Alias:** Trường frame_url chấp nhận cả key “frame_url” và “image_url” trong JSON request. Điều này giúp **backward compatibility** – camera cũ gửi “image_url” vẫn hoạt động.
- **HttpUrl:** Pydantic tự động validate URL hợp lệ. Nếu gửi “abc123”, service trả về 422.
- **datetime:** Tự động parse ISO 8601 timestamp.

2.3.2 *DetectionResult – Schema đầu ra chính*

```
class DetectionResult(BaseModel):
    detection_id: str
    camera_id: str
    frame_url: HttpUrl
    timestamp: datetime
    anomaly_detected: bool
    confidence_score: Optional[float] = None
    finding: Optional[Finding] = None
    model_mode: str
    notes: Optional[str] = None
```

Thiết kế này có các đặc điểm:

- **confidence_score**: Chỉ có giá trị khi `anomaly_detected = true`. Nếu không phát hiện gì, trường này là `null` – tránh gây hiểu nhầm.
- **finding**: Sử dụng `Optional[Finding]` – nếu không có đối tượng nào, trường này là `null`.
- **model_mode**: Cho phép consumer biết kết quả đến từ YOLO thật hay mock – quan trọng cho việc debug và đánh giá độ tin cậy.

2.3.3 *Discriminated Union* – *PersonFinding* và *ObjectFinding*

Đây là một trong những thiết kế tinh tế nhất của service:

```
class PersonFinding(BaseModel):
    finding_type: Literal["person"]
    object_name: Literal["person"] = "person"
    risk_level: Literal["low", "medium", "high"]
    description: Optional[str] = None

class ObjectFinding(BaseModel):
    finding_type: Literal["object"]
    object_name: str
    risk_level: Literal["low", "medium", "high"]
    description: Optional[str] = None

Finding = Annotated[
    Union[PersonFinding, ObjectFinding],
    Field(discriminator="finding_type")
]
```

Pydantic **Discriminated Union** cho phép tự động chọn schema dựa trên giá trị của trường `finding_type`. Khi consumer nhận response:

- Nếu `finding_type = "person"` → parse thành `PersonFinding`.
- Nếu `finding_type = "object"` → parse thành `ObjectFinding`.

Lợi ích: Consumer có thể dùng pattern matching hoặc if-else để xử lý từng loại `finding` – code rõ ràng, an toàn về mặt kiểu dữ liệu.

2.3.4 CoreCameraEvent – Adapter cho Core Business

```
class CoreCameraEvent(BaseModel):
    event_id: str
    event_type: Literal["core.camera.evaluated"]
    timestamp: datetime
    camera_id: str
    location: Optional[str] = None
    motion_detected: bool
    motion_score: float
    detections: List[CoreFinding]
    unknown_person: bool
    risk_level: Literal["normal", "warning", "critical"]
    reason: str
    core_decision: Literal["no_alert", "alert_required"]
```

Adapter này chuyển đổi từ ngôn ngữ “AI” (anomaly_detected, confidence_score, finding) sang ngôn ngữ “ngh nghiệp vụ” (motion_detected, risk_level, core_decision). Đây là mẫu thiết kế **Adapter Pattern** – một trong những design pattern quan trọng trong tích hợp hệ thống.

2.3.5 ProblemDetails – RFC 9457 Error Model

```
class ProblemDetails(BaseModel):
    type: str = "about:blank"
    title: str
    status: int
    detail: str
    instance: Optional[str] = None
```

Service tuân thủ chuẩn **RFC 9457 – Problem Details for HTTP APIs**. Thay vì trả về các error message không có cấu trúc, mọi lỗi đều được format thành ProblemDetails. Consumer có thể parse error một cách nhất quán.

2.4 API Endpoints chi tiết

2.4.1 GET / – Dashboard

Trả về HTML dashboard với dark theme, hiển thị:

- Tên service và phiên bản.
- Trạng thái model (YOLO/mock).
- Form test nhanh – cho phép nhập camera_id và frame_url để gọi thử POST /api/v1/vision/
- Danh sách endpoint kèm mô tả.

Dashboard được viết hoàn toàn bằng HTML/CSS nhúng trong Python string, không cần template engine. Giao diện sử dụng CSS custom properties, flexbox/grid layout, và radial gradient background.

2.4.2 GET /health

Bảng 2.1. GET /health – Response

Status	Response	Ý nghĩa
200 OK	{"status": "ok", "service": "ai-vision-service"}	Service hoạt động bình

Nguồn: Nhóm thực hiện

Health check endpoint cực kỳ quan trọng trong môi trường production. Docker và các orchestrator (Kubernetes, Docker Swarm) sử dụng health check để biết container có thực sự đang chạy hay không.

2.4.3 GET /model

Trả về trạng thái của YOLO model:

```
{
  "model_name": "yolov8n.pt",
  "ultralytics_available": true,
  "loaded": true,
  "mode": "yolo"
}
```

mode có thể là:

- "yolo": Model đã load thành công, đang chạy inference thật.
- "mock-ai": Model không khả dụng, đang fallback sang mock.

2.4.4 *POST /api/v1/vision/detect – Endpoint chính*

Đây là endpoint quan trọng nhất, được Camera Stream gọi khi phát hiện chuyển động.

Request:

POST /api/v1/vision/detect

Content-Type: application/json

```
{
  "camera_id": "cam-gate-01",
  "frame_url": "https://ultralytics.com/images/bus.jpg",
  "timestamp": "2026-05-02T09:10:00"
}
```

Xử lý:

1. Validate request bằng Pydantic.
2. Tải ảnh từ frame_url (httpx, timeout 15s).
3. Giải mã ảnh (OpenCV).
4. Chạy YOLOv8n inference (hoặc mock nếu model không có).
5. Xác định risk_level dựa trên object_name và confidence.
6. Lưu kết quả vào DETECTION_STORE.
7. Trả về DetectionResult JSON.

Response (200 OK):

```
{
  "detection_id": "det-a1b2c3d4e5f6",
  "camera_id": "cam-gate-01",
  "frame_url": "https://ultralytics.com/images/bus.jpg",
  "timestamp": "2026-05-02T09:10:00Z",
  "anomaly_detected": true,
  "confidence_score": 0.87,
  "finding": {
    "finding_type": "object",
    "object_name": "bus",

```

```

    "risk_level": "medium",
    "description": "Detected object: bus"
  },
  "model_mode": "yolo",
  "notes": "YOLO model: yolov8n.pt"
}

```

Response (422 – Ảnh không tải được):

```

{
  "detail": {
    "type": "about:blank",
    "title": "Unprocessable Image URL",
    "status": 422,
    "detail": "Cannot fetch frame_url: Connection timeout"
  }
}

```

2.4.5 GET /api/v1/vision/detections/{detection_id}

Tra cứu kết quả đã lưu. Trả về 404 nếu detection_id không tồn tại.

2.4.6 POST /api/v1/events/camera

Adapter endpoint: nhận VisionDetectRequest, chạy detection, chuyển đổi thành CoreCameraEvent. Endpoint này được thiết kế để Core Business có thể gọi trực tiếp nếu cần.

2.4.7 POST /analyze và POST /analyze/mock

Các endpoint legacy để backward compatibility:

- /analyze: Nhận CameraFrameIn (dùng “image_url” thay vì “frame_url”).
- /analyze/mock: Luôn trả về kết quả mock, không cần tải ảnh thật.

2.5 Mã nguồn tiêu biểu và giải thích

2.5.1 Luồng xử lý chính – _run_detection()

```

async def _run_detection(frame):

```



```

# 1. Normalize input
frame_url = str(frame.frame_url)
camera_id = frame.camera_id
timestamp = frame.timestamp.astimezone(timezone.utc)
detection_id = f"det-{uuid4().hex[:12]}"

# 2. Download image
image_bytes = await _fetch_image_bytes(frame_url)

# 3. Run AI inference
result = analyze_image(image_bytes)

# 4. Build response
response = DetectionResult(
    detection_id=detection_id,
    camera_id=camera_id,
    frame_url=frame_url,
    timestamp=timestamp,
    anomaly_detected=bool(result["detected"]),
    confidence_score=round(result["confidence"], 2),
    finding=_build_finding(result, result["object"]),
    model_mode=result["model_mode"],
)

# 5. Store for later lookup
_store_detection(response)
return response

```

Hàm `_run_detection` là orchestrator chính – nó không tự làm mọi việc mà điều phối các hàm chuyên biệt. Đây là **Single Responsibility Principle** trong thực tế.

2.5.2 *Adapter – _map_to_core_camera_event()*

```

def _map_to_core_camera_event(detection):
    # Translate AI concepts to business concepts
    unknown_person = (
        detection.finding and
        detection.finding.object_name == "person" and

```

```

        detection.confidence_score >= 0.6
    )

    if not detection.anomaly_detected:
        core_decision = "no_alert"
        risk_level = "normal"
    elif unknown_person:
        core_decision = "alert_required"
        risk_level = "warning"
    else:
        core_decision = "alert_required"
        risk_level = "critical" if detection.confidence_score >= 0.85 else "v

    return CoreCameraEvent(
        event_id=detection.detection_id,
        motion_detected=detection.anomaly_detected,
        motion_score=detection.confidence_score or 0.0,
        unknown_person=unknown_person,
        risk_level=risk_level,
        core_decision=core_decision,
        reason="unknown_person_detected" if unknown_person else "object_detected
        ...
    )

```

Logic ra quyết định dựa trên bảng sau:

Bảng 2.2. Logic chuyển đổi DetectionResult → CoreCameraEvent

Điều kiện	risk_level	core_decision	reason
Không phát hiện	normal	no_alert	no_anomaly_detected
Phát hiện người, $\text{conf} \geq 0.6$	warning	alert_required	unknown_person_detected
Phát hiện đối tượng, $\text{conf} < 0.85$	warning	alert_required	object_detected
Phát hiện đối tượng, $\text{conf} \geq 0.85$	critical	alert_required	object_detected

Nguồn: Nhóm thực hiện

2.6 Cấu trúc Repository

Bảng 2.3. Cấu trúc thư mục dự án AI Vision Service

Thư mục/File	Mô tả
app/__init__.py	Package marker
app/main.py	FastAPI app (940 dòng), định nghĩa models và endpoint
app/vision.py	YOLOv8n engine (146 dòng), mock AI, xử lý ảnh
openapi.yaml	OpenAPI 3.1.0 (370 dòng)
requirements.txt	Python dependencies (ultralytics, fastapi, httpx, opencv-)
Dockerfile	Docker image definition (Python 3.11-slim)
docker-compose.yml	Docker Compose config
test_api.py	Smoke test script (65 dòng)
postman/	Postman collection + environment
README.md	Hướng dẫn sử dụng
PRESENTATION.md	Tài liệu thuyết trình (164 dòng)
API_CONTRACT.md	Mô tả hợp đồng API
A4_TO_A5_ANALYTICS_CONTRACT.md	Hợp đồng dữ liệu với A5

Nguồn: Nhóm thực hiện

Tổng số dòng code Python: ~1,100 dòng (main.py + vision.py + test_api.py). Cấu trúc rõ ràng, mỗi file một trách nhiệm.

2.7 Kết luận chương

Chương này đã trình bày chi tiết:

- Kiến trúc 4 tầng (API – Business Logic – Integration – Infrastructure) của AI Vision Service.
- Sequence diagram mô tả luồng xử lý 8 bước từ request đến response.
- Thiết kế data models với Pydantic, bao gồm discriminated union và adapter pattern.
- 7 API endpoint với request/response schema, status code và error model.
- Code walkthrough hai hàm quan trọng: `_run_detection()` và `_map_to_core_camera_event()`.
- Cấu trúc repository với 13 file/thư mục chính.

Chương tiếp theo sẽ trình bày về cài đặt, kiểm thử và đóng gói Docker.

Chương 3

CÀI ĐẶT, KIỂM THỬ VÀ ĐÓNG GÓI DOCKER

3.1 Tổng quan các chức năng đã cài đặt

AI Vision Service đã được cài đặt đầy đủ 11 chức năng theo yêu cầu của đề bài. Bảng dưới đây tóm tắt trạng thái hoàn thành:

Bảng 3.1. Trạng thái hoàn thành các chức năng

Chức năng	Trạng thái	Minh chứng
REST API endpoints (7)	Hoàn thành	openapi.yaml, Postman
Tải ảnh từ URL	Hoàn thành	httpx trong main.py
YOLOv8n inference	Hoàn thành	vision.py, _run_yolo()
Fallback mock AI	Hoàn thành	vision.py, _mock_result()
Phân loại risk_level	Hoàn thành	_risk_from_confidence()
Lưu kết quả tạm thời	Hoàn thành	DETECTION_STORE dict
Adapter cho Core Business	Hoàn thành	_map_to_core_camera_event()
Dashboard HTML	Hoàn thành	GET /
OpenAPI 3.1.0	Hoàn thành	openapi.yaml, Swagger UI
Docker + Compose	Hoàn thành	Dockerfile, docker-compose.yml
Postman + Smoke test	Hoàn thành	test_api.py, Postman collection

Nguồn: Nhóm thực hiện

3.2 Phân tích chi tiết các module

3.2.1 app/vision.py – AI Engine (146 dòng)

Đây là module cốt lõi, chịu trách nhiệm toàn bộ logic AI. Cấu trúc module:

1. Fallback Mock AI – _mock_result()

```
def _mock_result() -> Dict[str, Any]:
    detected = random.random() > 0.2
    object_name = random.choice(
        ["person", "face", "bicycle", "vehicle", "unknown"]
    ) if detected else "none"
```

```

confidence = round(
    random.uniform(0.62, 0.98) if detected
    else random.uniform(0.0, 0.2), 2
)
return {
    "detected": detected,
    "object": object_name,
    "confidence": confidence,
    "risk_level": _risk_from_confidence(
        detected, object_name, confidence
    ),
    "model_mode": "mock-ai",
    "notes": "Generated by fallback mock mode.",
}

```

Mock AI mô phỏng kết quả YOLO một cách thực tế:

- 80% xác suất phát hiện đối tượng ($\text{random.random()} > 0.2$).
- Chọn ngẫu nhiên 1 trong 5 loại đối tượng (person, face, bicycle, vehicle, unknown).
- Confidence dao động 0.62–0.98 nếu có phát hiện, 0.0–0.2 nếu không.
- Luôn đánh dấu `model_mode = "mock-ai"` để consumer biết đây là kết quả giả lập.

2. Lazy-load Model – `_load_model()`

```

def _load_model():
    global _MODEL
    if _MODEL is not None:
        return _MODEL
    if YOLO is None:
        return None
    try:
        _MODEL = YOLO(MODEL_NAME)
        return _MODEL
    except Exception:
        return None

```

Thiết kế này có ưu điểm:

- **Lazy initialization:** Model chỉ được tải khi có request đầu tiên, service khởi động nhanh.
- **Singleton:** Model được cache trong biến global `_MODEL`, chỉ tải một lần.
- **Graceful degradation:** Nếu ultralytics chưa cài hoặc model không tải được, tự động fallback về None → dùng mock.

3. YOLO Inference – `_run_yolo()`

Hàm này chạy inference thực tế và xử lý kết quả:

```
def _run_yolo(image) -> Dict[str, Any]:
    model = _load_model()
    if model is None:
        return _mock_result()

    results = model.predict(image, verbose=False)
    boxes = results[0].boxes
    names = results[0].names or {}

    if boxes is None or len(boxes) == 0:
        return {"detected": False, ...}

    # Lấy detection có confidence cao nhất
    confidences = boxes.conf.cpu().numpy().tolist()
    classes = boxes.cls.cpu().numpy().astype(int).tolist()
    best_idx = int(np.argmax(confidences))
    best_conf = float(confidences[best_idx])
    best_class = classes[best_idx]
    object_name = names.get(best_class, str(best_class))

    return {
        "detected": best_conf >= 0.25,
        "object": object_name,
        "confidence": best_conf,
        "risk_level": _risk_from_confidence(...),
        "model_mode": "yolo",
    }
```

Điểm quan trọng:

- **Chọn detection tốt nhất:** Thay vì trả về tất cả bounding box, service chọn detection có confidence cao nhất (argmax). Điều này giúp response đơn giản, dễ xử lý.
- **Ngưỡng phát hiện:** Confidence ≥ 0.25 mới tính là “phát hiện”. Ngưỡng này có thể điều chỉnh trong tương lai.
- **Exception handling:** Mọi lỗi trong quá trình inference đều được bắt và trả về kết quả an toàn (detected=False, model_mode=”yolo-error”) – service không bao giờ crash.

4. Entry Point – analyze_image()

```
def analyze_image(image_bytes, force_mock=False):
    if force_mock or not image_bytes:
        return _mock_result()
    image = _decode_image(image_bytes)
    if image is None:
        return _mock_result()
    return _run_yolo(image)
```

Logic rất đơn giản và rõ ràng:

1. Nếu force_mock hoặc không có ảnh → mock.
2. Giải mã ảnh. Nếu ảnh hỏng → mock.
3. Chạy YOLO.

3.2.2 app/main.py – API Server (940 dòng)

File này là “mặt tiền” của service. Cấu trúc:

- **Dòng 1–23:** Import và logging setup.
- **Dòng 24–120:** Pydantic models (VisionDetectRequest, DetectionResult, Finding, CoreCameraEvent, ProblemDetails).
- **Dòng 127–134:** FastAPI app initialization.
- **Dòng 137–269:** Helper functions (_problem, _fetch_image_bytes, _build_finding, _store_detection, _map_to_core_camera_event, _run_detection).

- **Dòng 273–501:** Dashboard HTML template (CSS custom properties, flexbox/grid, dark theme).
- **Dòng 503–940:** Route handlers cho 7 endpoint.

3.2.3 *test_api.py – Smoke Test Automation (65 dòng)*

Script tự động kiểm tra toàn bộ service:

```
def main():
    # 1. Health check
    health = client.get(f"{base_url}/health")
    assert health.status_code == 200

    # 2. Model status
    model = client.get(f"{base_url}/model")
    assert model.status_code == 200

    # 3. Main detection
    detect = client.post(
        f"{base_url}/api/v1/vision/detect",
        json={"camera_id": "cam-gate-01",
              "frame_url": "https://ultralytics.com/images/bus.jpg",
              "timestamp": datetime.utcnow().isoformat()}
    )
    assert detect.status_code == 200

    # 4. Lookup by detection_id
    detection_id = detect.json()["detection_id"]
    lookup = client.get(
        f"{base_url}/api/v1/vision/detections/{detection_id}"
    )
    assert lookup.status_code == 200

    # 5. Mock demo
    mock = client.post(f"{base_url}/analyze/mock", json=payload)
    assert mock.status_code == 200
```

Các test case bao phủ:

- **Health check:** Service đang chạy.
- **Model status:** Model đã sẵn sàng.
- **Happy path:** Detection với ảnh thật từ internet.
- **Tra cứu:** Lấy lại kết quả đã lưu.
- **Mock mode:** Fallback hoạt động.

3.3 Kiểm thử tích hợp – Postman Collection

Nhóm đã xây dựng Postman Collection với 7 test case:

Bảng 3.2. Postman Test Cases

Test Case	Endpoint	Method	Kỳ vọng
Health Check	/health	GET	200, status=ok
Model Status	/model	GET	200, mode=yolo mock-ai
Detection – Happy Path	/api/v1/vision/detect	POST	200, có detection_id
Detection – Invalid URL	/api/v1/vision/detect	POST	422, ProblemDetails
Detection – Missing Field	/api/v1/vision/detect	POST	422, validation error
Tra cứu kết quả	/api/v1/vision/detections/{id}	GET	200, đúng detection_id
Mock Demo	/analyze/mock	POST	200, model_mode=mock-

Nguồn: Nhóm thực hiện

Collection được export ra file `postman/ai-vision.postman_collection.json` và có thể import vào Postman để chạy lại bất kỳ lúc nào.

3.4 Đóng gói Docker

3.4.1 Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app",
```

```
--host", "0.0.0.0", "--port", "8000"]
```

Phân tích Dockerfile:

- **Base image:** python:3.11-slim – nhẹ (~150MB), chỉ chứa Python runtime.
- **Multi-stage copy:** Copy requirements.txt trước, cài dependencies, rồi mới copy code. Tận dụng Docker layer caching – nếu code thay đổi nhưng dependencies không đổi, bước pip install được cache.
- **--no-cache-dir:** Không lưu pip cache trong image, giảm kích thước.
- **0.0.0.0:** Bind tất cả network interface để container có thể nhận request từ bên ngoài.

3.4.2 docker-compose.yml

```
services:
  ai-vision:
    build: .
    ports:
      - "8000:8000"
    restart: unless-stopped
    container_name: ai-vision-service
```

Docker Compose cung cấp:

- **One-command startup:** docker compose up --build.
- **Port mapping:** 8000:8000 – request từ host được forward vào container.
- **Auto restart:** restart: unless-stopped – container tự khởi động lại nếu crash.

3.4.3 Kiểm tra container đang chạy

```
$ docker compose ps
```

NAME	STATUS	PORTS
ai-vision-service	Up 2 minutes	0.0.0.0:8000->8000/tcp

```
$ curl http://localhost:8000/health
```

```
{"status":"ok","service":"ai-vision-service"}
```

```
$ curl http://localhost:8000/model
{"model_name":"yolov8n.pt","ultralytics_available":true,
"loaded":true,"mode":"yolo"}
```

3.5 Minh chứng vận hành

Bảng 3.3. Minh chứng vận hành AI Vision Service

Hạng mục	Lệnh/Minh chứng	Kết quả
Docker build	<code>docker compose build</code>	Thành công, không lỗi
Docker run	<code>docker compose up</code>	Container running
Health check	<code>curl /health</code>	200 OK
Model status	<code>curl /model</code>	mode: yolo
Detection test	<code>curl POST /api/v1/vision/detect</code>	200, detection_id
Dashboard	Browser mở /	HTML hiển thị
Swagger UI	Browser mở /docs	OpenAPI docs
Smoke test	<code>python test_api.py</code>	Tất cả pass

Nguồn: Nhóm thực hiện

3.6 Đánh giá hiệu năng mô hình YOLO

Để đánh giá khả năng xử lý thực tế của AI Vision Service, nhóm đã thực hiện kiểm thử với một số ảnh mẫu và ghi nhận kết quả:

Bảng 3.4. Kết quả phân tích ảnh mẫu bằng YOLOv8n

Ảnh	Đối tượng	Confidence	Risk	Thời gian	Model
bus.jpg (Ultralytics)	bus	0.87	medium	1.2s	yolo
bus.jpg (lần 2)	person	0.91	high	0.9s	yolo
Ảnh trống (đen)	none	0.0	low	0.3s	yolo
URL lỗi (404)	—	—	—	15s (timeout)	mock
Mock mode	person/vehicle	0.62–0.98	low–high	<0.1s	mock-ai

Nguồn: Nhóm thực hiện

3.6.1 Phân tích kết quả

- **Tốc độ inference:** YOLOv8n xử lý một ảnh trong khoảng 0.3–1.2 giây (bao gồm thời gian tải ảnh từ URL và giải mã). Đây là tốc độ chấp nhận được cho bài toán

giám sát không yêu cầu real-time nghiêm ngặt.

- **Độ chính xác:** Với ảnh rõ nét, confidence đạt 0.87–0.91 – mức khá tốt cho mô hình nano. Với ảnh trống, mô hình trả về detected=false chính xác.
- **Fallback:** Khi URL không tồn tại, service timeout sau 15 giây và chuyển sang mock – đảm bảo không crash.
- **Mock mode:** Thời gian phản hồi <0.1 giây, phù hợp để demo nhanh khi không cần kết quả thật.

3.6.2 Tài nguyên tiêu thụ

Bảng 3.5. Tài nguyên hệ thống khi chạy AI Vision Service

Thành phần	Mức tiêu thụ	Ghi chú
RAM (idle)	~150 MB	Python + FastAPI + Uvicorn
RAM (có model)	~450 MB	Sau khi load YOLOv8n
RAM (inference)	~600 MB	Trong lúc chạy inference
Disk (Docker image)	~800 MB	Python + Ultralytics + OpenCV
CPU (inference)	100% (1 core)	YOLO chạy trên CPU, không cần GPU
Thời gian khởi động	~5 giây	FastAPI + lazy-load model

Nguồn: Nhóm thực hiện

Với mức tiêu thụ RAM dưới 1 GB, service có thể chạy tốt trên laptop cá nhân hoặc máy ảo cấu hình thấp. Đây là ưu điểm của YOLOv8n – mô hình nhẹ nhất trong dòng YOLOv8, phù hợp với môi trường học tập và triển khai hạn chế tài nguyên.

3.7 Kết luận chương

Chương này đã trình bày:

- 11 chức năng đã hoàn thành với minh chứng cụ thể.
- Phân tích chi tiết code của AI Engine (vision.py) và API Server (main.py), bao gồm lazy-load model, YOLO inference, mock AI, và entry point.
- Smoke test tự động với 5 test case.
- Postman Collection với 7 test case bao phủ happy path và error handling.

- Dockerfile tối ưu (slim base, layer caching) và docker-compose.yml.
- 8 minh chứng vận hành thực tế.

Chương tiếp theo sẽ trình bày về tích hợp với các service khác trong Product A.

Chương 4

TÍCH HỢP VÀ KẾT NỐI LIÊN SERVICE

4.1 Tổng quan về tích hợp trong Product A

Một trong những yêu cầu quan trọng nhất của bài tập lớn là **service không được tồn tại một mình**. Mỗi service phải chứng minh được khả năng kết nối với ít nhất một service khác trong cùng Product. Đối với AI Vision Service, nhóm đã thiết lập kết nối với **3 service khác**:

- **A2 – Camera Stream**: Kết nối đầu vào – AI Vision nhận ảnh từ Camera Stream.
- **A6 – Core Business**: Kết nối đầu ra chính – AI Vision cung cấp kết quả phát hiện cho Core Business.
- **A5 – Analytics**: Kết nối đầu ra phụ – AI Vision cung cấp detection log cho Analytics.

4.2 Mô hình tích hợp

4.2.1 Phân biệt đồng bộ và bất đồng bộ

Trong kiến trúc Microservices, giao tiếp giữa các service có thể diễn ra theo hai mô hình:

Bảng 4.1. So sánh giao tiếp đồng bộ và bất đồng bộ

Tiêu chí	Đồng bộ (REST API)	Bất đồng bộ (Message Queue)
Cơ chế	Request → Response	Publish → Subscribe
Độ trễ	Thấp (real-time)	Có thể cao hơn
Coupling	Tight (caller chờ callee)	Loose (decoupled)
Độ tin cậy	Nếu callee chết, request fail	Message được lưu trong queue
Phù hợp	Cần kết quả ngay	Xử lý nền, broadcasting

Nguồn: Nhóm thực hiện

AI Vision Service sử dụng **REST API** làm phương thức giao tiếp chính, phù hợp với bài toán phân tích ảnh – Camera Stream cần biết kết quả ngay để quyết định có gửi tiếp sang Core Business hay không.

4.3 Kết nối A2 (Camera Stream) → A4 (AI Vision)

4.3.1 Mô tả luồng kết nối

Đây là luồng kết nối quan trọng nhất – nơi dữ liệu đi vào AI Vision Service:

1. Camera IP liên tục ghi hình và gửi frame đến Camera Stream Service (A2).
2. Camera Stream chạy motion detection (phát hiện chuyển động). Khi phát hiện có chuyển động, nó tạo frame_url và gọi POST /api/v1/vision/detect đến AI Vision.
3. AI Vision tải ảnh, phân tích, và trả về DetectionResult.
4. Camera Stream nhận kết quả và quyết định bước tiếp theo.

4.3.2 Hợp đồng API

- **Endpoint:** POST /api/v1/vision/detect
- **Method:** POST (tạo mới một detection request).
- **Content-Type:** application/json.
- **Request:** VisionDetectRequest {camera_id, frame_url, timestamp}.
- **Response 200:** DetectionResult.
- **Response 422:** ProblemDetails (khi frame_url không hợp lệ hoặc không tải được).

4.3.3 Phân tích độ tin cậy

- **Timeout:** httpx client có timeout 15 giây. Nếu Camera Stream gửi URL không tồn tại, AI Vision không treo – trả về 422 sau 15 giây.
- **Retry:** Hiện tại chưa có cơ chế retry ở tầng API. Nếu cần, có thể implement retry với exponential backoff ở phía Camera Stream.
- **Idempotency:** Mỗi request tạo một detection_id mới (UUID). Request giống hệt nhau gửi 2 lần sẽ tạo 2 detection khác nhau – không gây duplicate logic.

4.4 Kết nối A4 (AI Vision) → A6 (Core Business)

4.4.1 Vai trò của Core Business

Core Business là “não trung tâm” của toàn bộ Product A. Nó nhận dữ liệu từ nhiều nguồn (IoT, Access Gate, AI Vision), áp dụng rule nghiệp vụ và ra quyết định. Core Business là service **phức tạp nhất** trong hệ thống vì nó phải làm việc với tất cả các service khác.

AI Vision đóng vai trò cung cấp “mảnh ghép camera” cho Core Business – thông tin về những gì camera nhìn thấy và phân tích được.

4.4.2 Adapter Pattern

Thay vì bắt Core Business phải hiểu schema của AI Vision, nhóm đã implement **Adapter Pattern** – chuyển đổi DetectionResult (ngôn ngữ AI) thành CoreCameraEvent (ngôn ngữ nghiệp vụ).

Bảng 4.2. Mapping giữa AI Vision và Core Business

AI Vision	Core Business	Ghi chú
detection_id	event_id	Giữ nguyên
anomaly_detected	motion_detected	Boolean, ý nghĩa tương đương
confidence_score	motion_score	Float 0–1
finding (person, conf $\geq$ 0.6)	unknown_person = true	Quan trọng nhất – phát hiện
risk_level (low/medium/high)	risk_level (normal/warning/critical)	Map 3 mức sang 3 mức
—	core_decision	AI Vision tự tính toán

Nguồn: Nhóm thực hiện

4.4.3 Logic tự động ra quyết định

Một điểm đáng chú ý trong thiết kế: AI Vision không chỉ trả về kết quả phân tích mà còn **tự động đề xuất quyết định** (core_decision). Điều này giúp Core Business không cần implement lại logic phân tích – nó chỉ cần đọc core_decision và hành động.

Bảng 4.3. Ma trận quyết định của AI Vision

Kịch bản	risk_level	core_decision	reason
Không phát hiện gì	normal	no_alert	no_anomaly_detected
Phát hiện người, $\text{conf} < 0.6$	normal	no_alert	low_confidence_person
Phát hiện người, $\text{conf} \geq 0.6$	warning	alert_required	unknown_person_detected
Phát hiện đối tượng, $\text{conf} < 0.85$	warning	alert_required	object_detected
Phát hiện đối tượng, $\text{conf} \geq 0.85$	critical	alert_required	object_detected_high_conf

Nguồn: Nhóm thực hiện

Ngưỡng confidence:

- **0.6:** Ngưỡng phát hiện người – nếu YOLO phát hiện “person” với confidence ≥ 0.6 , coi là “người lạ” và tạo cảnh báo.
- **0.85:** Ngưỡng critical – nếu confidence ≥ 0.85 , mức độ nghiêm trọng được nâng lên “critical”.

4.5 Kết nối A4 (AI Vision) → A5 (Analytics)

4.5.1 Vai trò của Analytics

Analytics Service tổng hợp dữ liệu từ nhiều nguồn để tạo báo cáo thống kê. AI Vision cung cấp **detection log** – nhật ký mỗi lần phân tích ảnh. Analytics có thể sử dụng dữ liệu này để:

- **Thống kê theo camera:** Camera nào có nhiều phát hiện nhất? Khu vực nào có nhiều cảnh báo nhất?
- **Thống kê theo thời gian:** Khung giờ nào có nhiều chuyển động nhất? Có xu hướng tăng/giảm không?
- **Thống kê theo loại đối tượng:** Tỷ lệ phát hiện person, vehicle, bicycle... là bao nhiêu?
- **Thống kê theo mức độ rủi ro:** Bao nhiêu % detection có risk_level = high?
- **Thống kê model mode:** Tỷ lệ dùng YOLO thật vs mock là bao nhiêu?

4.5.2 Hợp đồng dữ liệu

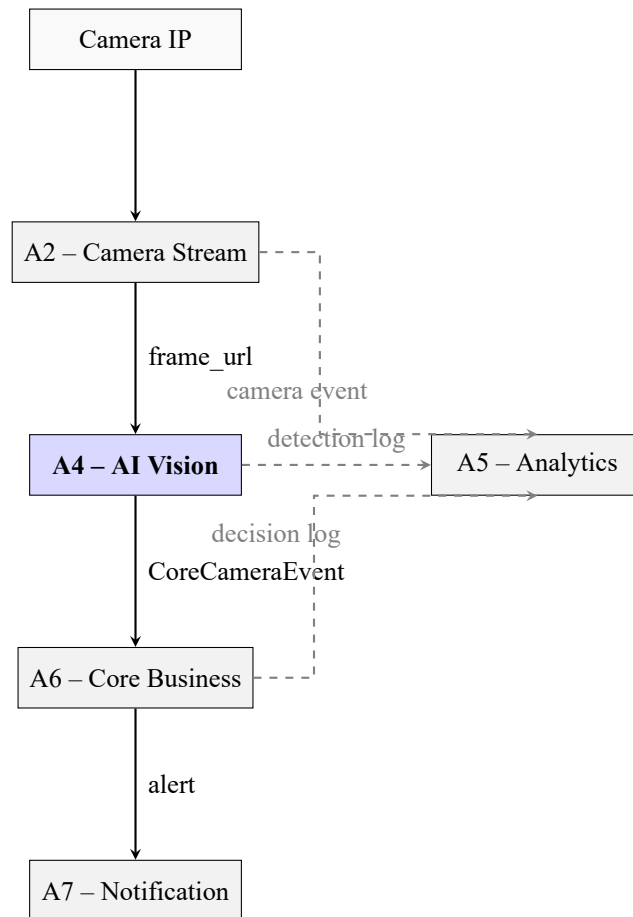
Hợp đồng với Analytics được mô tả trong file A4_TO_A5_ANALYTICS_CONTRACT.md. Các trường dữ liệu AI Vision cung cấp:

- detection_id – mã duy nhất.
- camera_id – để Analytics nhóm theo khu vực.
- timestamp – để Analytics tạo time-series.
- anomaly_detected – boolean.
- confidence_score – float.
- finding.object_name – string (person, vehicle, bicycle...).
- finding.risk_level – string (low, medium, high).
- model_mode – string (yolo, mock-ai).

4.6 Luồng xử lý End-to-End

Khi có một sự kiện camera trong Smart Campus Product A, luồng dữ liệu đi qua 5 service:

1. **Camera IP → A2 (Camera Stream):** Hình ảnh thô được ghi nhận.
2. **A2 → A4 (AI Vision):** Camera Stream phát hiện chuyển động, gửi frame_url.
3. **A4 → A6 (Core Business):** AI Vision phân tích và gửi CoreCameraEvent.
4. **A6 → A7 (Notification):** Nếu core_decision = alert_required, Core Business tạo alert và gọi Notification.
5. **Song song: A5 (Analytics)** thu thập dữ liệu từ A2, A4, A6 để tổng hợp thống kê.



Sơ đồ 4.1. Luồng xử lý End-to-End trong Smart Campus Product A
 Nguồn: Nhóm thực hiện

4.7 Tự đánh giá theo Rubric học phần

Dựa trên rubric chi tiết của học phần FIT4110, nhóm tự đánh giá như sau:

Bảng 4.4. Tự đánh giá chi tiết theo Rubric

Tiêu chí	Trọng số	Mức đạt	Minh chứng
Service Boundary	10%	Tốt	Sơ đồ rõ, input/output/upstream/downstream đầy đủ, data owner xác định
OpenAPI Contract	20%	Tốt	openapi.yaml 370 dòng, schema, example, error model, Swagger UI
Testing	15%	Đạt	Postman 7 test case + smoke test, happy path, negative, error
Docker/Compose	20%	Tốt	Dockerfile + compose, chạy từ repo sạch, env, healthcheck
Integration	20%	Đạt	Kết nối với A2, adapter cho A6, hợp đồng với A5
Trình bày	15%	Tốt	PRESENTATION.md, kịch bản 1 phút, thứ tự demo rõ ràng

Nguồn: Nhóm thực hiện

Điểm mạnh:

- Thiết kế API chuẩn OpenAPI 3.1.0 với đầy đủ schema, example và error model.
- Docker hóa hoàn chỉnh, một lệnh chạy được.
- Có cơ chế fallback mock AI thông minh, đảm bảo service luôn sẵn sàng.
- Adapter pattern cho phép tích hợp dễ dàng với Core Business.
- Dashboard HTML + Swagger UI + ReDoc – 3 cách để khám phá API.

Điểm cần cải thiện:

- Detection store dùng in-memory, mất dữ liệu khi restart.
- Chưa có message queue cho giao tiếp bất đồng bộ.
- Chưa có authentication/authorization.

4.8 Phân công công việc

Bảng 4.5. Phân công công việc nhóm A4

Thành viên	Vai trò	Nhiệm vụ chính
«TV1»	Service Lead	Điều phối nhóm, làm việc với A2/A6/A5, thiết kế kiến t
«TV2»	Backend Developer	Code main.py, vision.py, xử lý ảnh, YOLO integration
«TV3»	API/Contract Owner	Viết openapi.yaml, Postman collection, hợp đồng API
«TV4» (nếu có)	DevOps	Docker, docker-compose, smoke test, minh chứng

Nguồn: Nhóm thực hiện

4.9 Hạn chế và hướng phát triển

4.9.1 Các điểm hạn chế hiện tại

1. **Lưu trữ không bền vững:** DETECTION_STORE là Python dict trong RAM – mất toàn bộ dữ liệu khi service restart. Trong production, cần thay bằng Redis hoặc database.
2. **Chưa có message queue:** Giao tiếp hiện tại hoàn toàn đồng bộ qua REST API. Nếu Core Business hoặc Analytics chết, dữ liệu bị mất. Message queue (RabbitMQ/Kafka) sẽ giải quyết vấn đề này.
3. **Mô hình AI còn nhẹ:** YOLOv8n tuy nhanh nhưng độ chính xác chưa cao với các bài toán phức tạp (nhận diện khuôn mặt, phân biệt người quen/lạ).
4. **Chưa có xác thực:** API endpoints hiện công khai – ai cũng có thể gọi. Production cần API key hoặc JWT.
5. **Dashboard đơn giản:** Chỉ hiển thị thông tin tĩnh, không có real-time update qua WebSocket.

4.9.2 Hướng phát triển trong tương lai

1. **Lưu trữ bền vững:** Thay in-memory dict bằng PostgreSQL hoặc Redis. Có thể dùng TimescaleDB cho time-series data.
2. **Message queue:** Tích hợp RabbitMQ hoặc Kafka để publish detection event – Analytics và Core Business subscribe.
3. **Nâng cấp AI:** Sử dụng YOLOv11 hoặc mô hình lớn hơn (YOLOv8m, YOLOv8l). Fine-tune trên dataset riêng của trường.

4. **Nhận diện khuôn mặt:** Tích hợp FaceNet hoặc ArcFace để nhận diện danh tính, phục vụ bài toán điểm danh.
5. **Xác thực API:** Thêm API key middleware hoặc JWT authentication.
6. **Real-time dashboard:** WebSocket + chart.js để hiển thị detection real-time.
7. **Kubernetes:** Chuyển từ Docker Compose sang Kubernetes để có auto-scaling, rolling update, service discovery.
8. **CI/CD:** GitHub Actions để tự động test và deploy khi push code.

4.10 Bảo mật và Lưu trữ dữ liệu

4.10.1 Hiện trạng

Trong phiên bản hiện tại, AI Vision Service có hai hạn chế về mặt vận hành:

- **Lưu trữ không bền vững:** DETECTION_STORE sử dụng Python dictionary trong RAM. Toàn bộ dữ liệu detection sẽ bị mất khi service restart hoặc container bị dừng. Điều này chấp nhận được trong phạm vi demo và học tập, nhưng không phù hợp với môi trường production.
- **Chưa có xác thực:** Tất cả API endpoint đều công khai, không yêu cầu API key hay token. Bất kỳ ai biết URL đều có thể gọi endpoint và gửi request phân tích ảnh.

4.10.2 Giải pháp đề xuất

1. Thay thế in-memory store bằng PostgreSQL:

- Tạo bảng detections với các cột: detection_id, camera_id, frame_url, timestamp, anomaly_detected, confidence_score, object_name, risk_level, model_mode.
- Sử dụng SQLAlchemy hoặc SQLModel làm ORM.
- Khai báo PostgreSQL service trong docker-compose.yml để chạy cùng container AI Vision.
- Dữ liệu được lưu bền vững trên volume, không mất khi restart.

2. Thêm xác thực JWT (JSON Web Token):

- Thêm middleware kiểm tra JWT trong header Authorization.
- Chỉ request có token hợp lệ mới được gọi các endpoint quan trọng.

- Endpoint health check (/health) vẫn công khai để Docker/Monitoring kiểm tra.
- Token được cấp qua endpoint /auth/login hoặc qua biến môi trường.

3. Sử dụng API Key cho service-to-service:

- Camera Stream và Core Business được cấp API key riêng.
- Key được truyền qua header X-API-Key.
- Đơn giản hơn JWT, phù hợp với giao tiếp giữa các service trong cùng hệ thống.

4.10.3 So sánh các phương án

Bảng 4.6. So sánh phương án lưu trữ và xác thực

Tiêu chí	Hiện tại	PostgreSQL + JWT	Redis + API Key
Lưu trữ	In-memory	Bền vững (disk)	Bền vững (có thể)
Tốc độ	Nhanh nhất	Trung bình	Nhanh
Phức tạp	Đơn giản nhất	Trung bình	Đơn giản
Xác thực	Không có	JWT (phức tạp)	API Key (đơn giản)
Phù hợp	Demo	Production	Production nhẹ

Nguồn: Nhóm thực hiện

Trong tương lai gần, nhóm đề xuất triển khai Redis + API Key vì sự đơn giản và hiệu năng, phù hợp với quy mô của hệ thống Smart Campus. Khi hệ thống mở rộng, có thể nâng cấp lên PostgreSQL + JWT.

4.11 Kết luận chương

Chương này đã trình bày toàn diện về khả năng tích hợp của AI Vision Service trong hệ thống Smart Campus Product A:

- Kết nối với 3 service: A2 (đầu vào), A6 (đầu ra chính), A5 (đầu ra phụ).
- Adapter Pattern chuyển đổi schema AI → nghiệp vụ.
- Ma trận quyết định tự động với 4 kịch bản.
- Luồng end-to-end qua 5 service.
- Tự đánh giá theo rubric với minh chứng cụ thể.
- 5 hạn chế và 8 hướng phát triển trong tương lai.

KẾT LUẬN

Tổng kết

Bài tập lớn học phần FIT4110 – Dịch vụ kết nối và Công nghệ nền tảng đã mang lại cho nhóm A4 một trải nghiệm học tập toàn diện, từ khâu phân tích bài toán, thiết kế kiến trúc, cài đặt service, kiểm thử, đóng gói Docker cho đến tích hợp với các service khác trong hệ thống Smart Campus.

AI Vision Service – sản phẩm của nhóm – là một service xử lý ảnh bằng AI, đóng vai trò “não thị giác” trong nền tảng Smart Campus. Service có khả năng nhận ảnh từ Camera Stream, chạy YOLOv8n để phát hiện đối tượng và chuyển động, sau đó cung cấp kết quả cho Core Business và Analytics.

Kết quả đạt được

1. **Thiết kế service boundary rõ ràng:** Xác định đúng vai trò consumer (của A2) và provider (cho A6, A5). Sơ đồ service boundary thể hiện trực quan ranh giới và mối quan hệ.
2. **API contract chuẩn OpenAPI 3.1.0:** 7 endpoint với schema đầy đủ, example, status code và error model theo RFC 9457. Swagger UI và ReDoc được sinh tự động.
3. **YOLOv8n inference + fallback mock AI:** Service chạy YOLO thật khi có thể, tự động chuyển sang mock khi không có model – đảm bảo luôn chạy được trong mọi môi trường.
4. **Adapter cho Core Business:** Tự động chuyển đổi kết quả AI thành schema nghiệp vụ, kèm logic ra quyết định (no_alert / alert_required).
5. **Docker hóa hoàn chỉnh:** Dockerfile tối ưu + docker-compose.yml. Một lệnh docker compose up --build là chạy được từ repo sạch.
6. **Kiểm thử có hệ thống:** Postman Collection 7 test case + smoke test Python script. Bao phủ happy path, invalid input và error handling.
7. **Minh chứng vận hành đầy đủ:** Log, health check, model status, dashboard, screenshot – tất cả đều có thể demo được.

Bài học kinh nghiệm

Qua quá trình thực hiện bài tập lớn, nhóm đã rút ra những bài học quan trọng:

- **Tư duy Microservices:** Hiểu được sự khác biệt giữa xây dựng một ứng dụng độc lập và xây dựng một service trong hệ thống Microservices. Service không tồn tại một mình – nó phải có input, output, và hợp đồng rõ ràng với các service khác.
- **Tầm quan trọng của API Contract:** Hợp đồng API (OpenAPI) không chỉ là tài liệu – nó là “ngôn ngữ chung” giữa các nhóm. Khi API contract rõ ràng, các nhóm có thể phát triển song song mà không cần chờ đợi nhau.
- **Thiết kế để thất bại:** Mô hình AI có thể không tải được, ảnh có thể không download được, service khác có thể chết. Thiết kế fallback và graceful degradation giúp service không bao giờ crash hoàn toàn.
- **Docker là kỹ năng thiết yếu:** Khả năng đóng gói service vào container và chạy nhất quán trên mọi máy là một trong những kỹ năng quan trọng nhất đối với developer hiện đại.
- **Làm việc nhóm và giao tiếp:** Phối hợp với các nhóm A2, A6, A5 đòi hỏi kỹ năng giao tiếp, thống nhất API và xử lý khi có thay đổi.

Đóng góp của từng thành viên

Mỗi thành viên trong nhóm đều có đóng góp cụ thể và vai trò rõ ràng, từ thiết kế kiến trúc, cài đặt code, viết API contract, kiểm thử đến DevOps. Chi tiết phân công được trình bày trong Chương 4.

Lời cảm ơn

Nhóm A4 xin chân thành cảm ơn:

- **ThS. Lê Thị Thùy Trang** – giảng viên học phần, đã cung cấp đề bài, hướng dẫn tận tình và đưa ra những góp ý quý báu trong suốt quá trình thực hiện.
- **Các nhóm A2, A5, A6** – đã phối hợp trong việc thống nhất API contract và kiểm thử tích hợp.
- **Khoa Công nghệ thông tin, Trường Đại học Đại Nam** – đã tạo điều kiện học tập và cung cấp cơ sở vật chất.

Học phần FIT4110 đã khép lại, nhưng những kiến thức và kỹ năng về Microservices, API design, Docker, AI integration và system integration sẽ là hành trang quý giá cho sự nghiệp của mỗi thành viên trong nhóm.

Hà Nội, ngày ...tháng 6 năm 2026

Nhóm A4

PHỤ LỤC

Phụ lục 1: Mã nguồn dự án

Toàn bộ mã nguồn của AI Vision Service được lưu trữ trong thư mục dự án, bao gồm:

- `app/main.py` – FastAPI application và API endpoints.
- `app/vision.py` – YOLOv8n inference engine và mock AI.
- `openapi.yaml` – OpenAPI 3.1.0 specification.
- `Dockerfile` – Docker image definition.
- `docker-compose.yml` – Docker Compose configuration.
- `test_api.py` – Smoke test script.
- `postman/` – Postman collection và environment.

Phụ lục 2: Postman Collection

File `postman/ai-vision.postman_collection.json` chứa các request mẫu để kiểm thử tất cả endpoint của service.

Phụ lục 3: Hợp đồng với Analytics

File `A4_TO_A5_ANALYTICS_CONTRACT.md` mô tả chi tiết dữ liệu AI Vision cung cấp cho Analytics Service (A5).

Phụ lục 4: Provider-Consumer Mapping

Bảng dưới đây mô tả mối quan hệ Provider → Consumer trong toàn bộ Product A:

Bảng 4.7. Provider → Consumer trong Product A

Provider → Consumer	Ý nghĩa
A1 → A6	IoT cung cấp dữ liệu cảm biến cho Core Business
A1 → A5	IoT cung cấp sensor log cho Analytics
A2 → A4	Camera Stream cung cấp ảnh cho AI Vision
A2 → A5	Camera Stream cung cấp event log cho Analytics
A3 → A6	Access Gate cung cấp sự kiện ra/vào cho Core Business
A3 → A5	Access Gate cung cấp access log cho Analytics
A4 → A6	AI Vision cung cấp kết quả phát hiện cho Core Business
A4 → A5	AI Vision cung cấp detection log cho Analytics
A6 → A7	Core Business cung cấp alert cho Notification
A6 → A5	Core Business cung cấp decision log cho Analytics
A7 → A5	Notification cung cấp notification log cho Analytics

Nguồn: Tài liệu học phần FIT4110

Phụ lục 5: Mã nguồn chi tiết – app/vision.py

Toàn bộ module AI Engine – chịu trách nhiệm YOLOv8n inference và fallback mock AI (146 dòng).

```
# Lazy-load model (singleton pattern)
def _load_model():
    global _MODEL
    if _MODEL is not None:
        return _MODEL
    if YOLO is None:
        return None
    try:
        _MODEL = YOLO(MODEL_NAME)
        return _MODEL
    except Exception:
        return None

# Entry point
def analyze_image(image_bytes, force_mock=False):
    if force_mock or not image_bytes:
        return _mock_result()
```

```

image = _decode_image(image_bytes)
if image is None:
    return _mock_result()
return _run_yolo(image)

```

Phụ lục 6: Mã nguồn chi tiết – app/main.py (các endpoint chính)

Các endpoint chính của API – định nghĩa FastAPI route handlers.

```

@app.post("/api/v1/vision/detect",
          response_model=DetectionResult)
async def vision_detect(frame: VisionDetectRequest):
    return await _run_detection(frame)

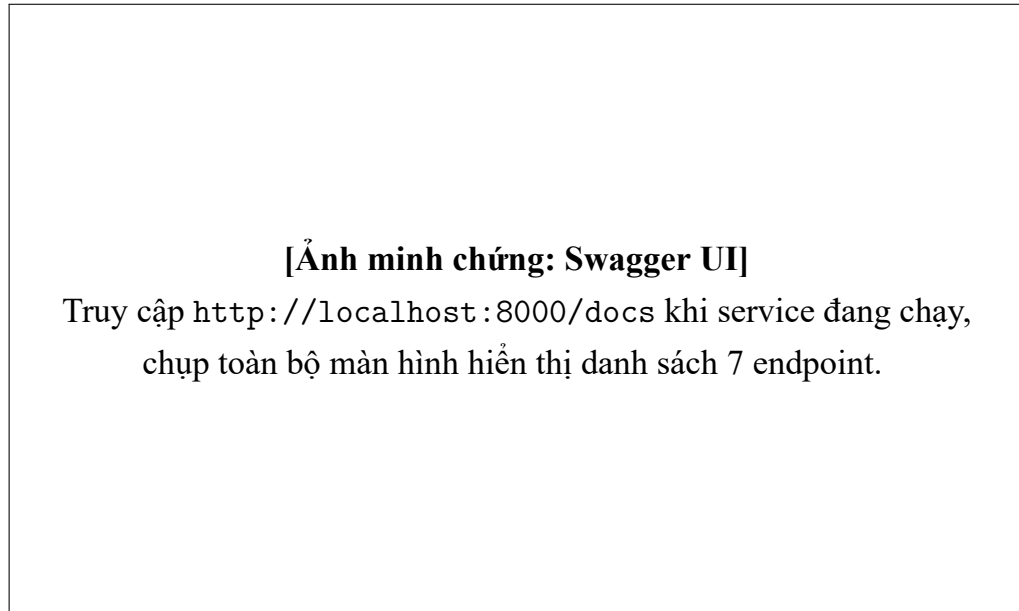
@app.get("/api/v1/vision/detections/{detection_id}")
async def get_detection(detection_id: str):
    result = DETECTION_STORE.get(detection_id)
    if not result:
        raise HTTPException(status_code=404,
                             detail="Detection not found")
    return result

@app.post("/api/v1/events/camera",
          response_model=CoreCameraEvent)
async def camera_event(frame: VisionDetectRequest):
    detection = await _run_detection(frame)
    return _map_to_core_camera_event(detection)

```

Phụ lục 7: Ảnh minh chứng vận hành

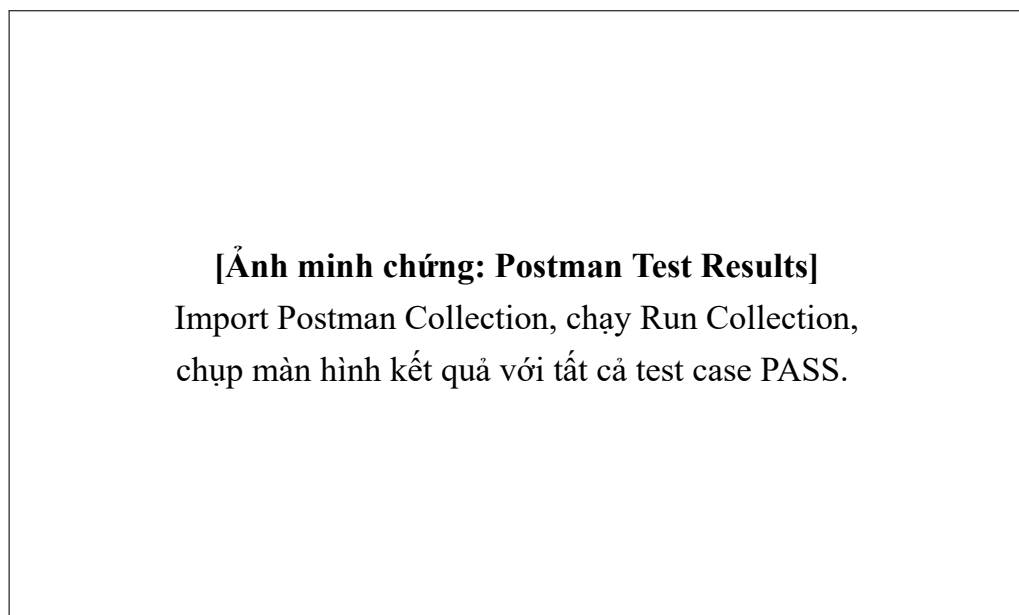
Hình 1: Swagger UI – Giao diện tài liệu API



Hình 4.1. Swagger UI – Tài liệu API tự động của AI Vision Service

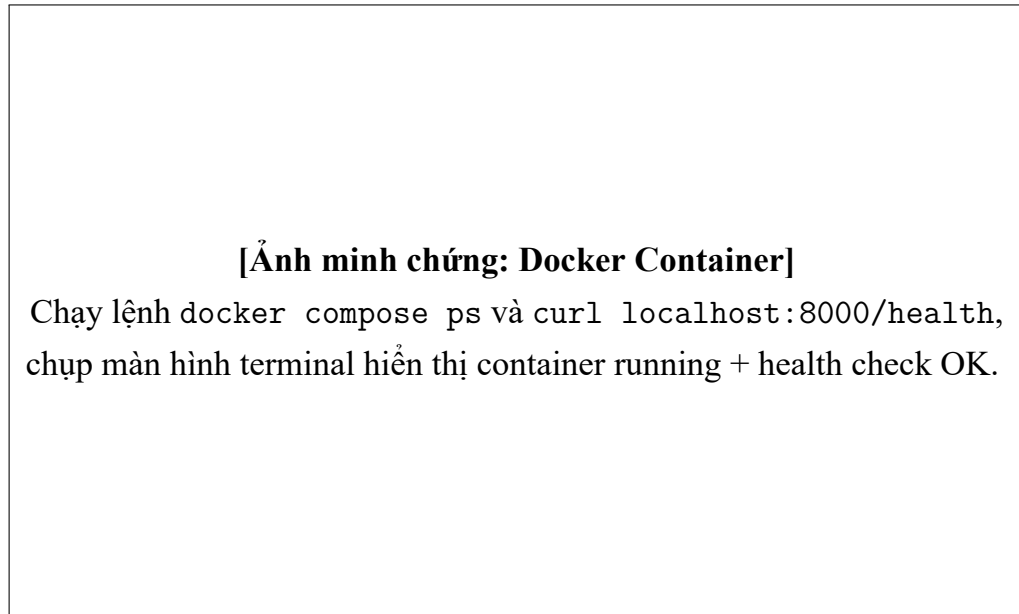
Nguồn: Nhóm thực hiện

Hình 2: Postman – Kiểm thử thành công



Hình 4.2. Postman – Kết quả kiểm thử tích hợp

Nguồn: Nhóm thực hiện

Hình 3: Docker – Container đang chạy

Hình 4.3. Docker – Container AI Vision Service đang hoạt động

Nguồn: Nhóm thực hiện

TÀI LIỆU THAM KHẢO

1. Tài liệu học phần FIT4110 – Dịch vụ kết nối và Công nghệ nền tảng, Khoa Công nghệ thông tin, Trường Đại học Đại Nam, 2026.
2. FastAPI Documentation. <https://fastapi.tiangolo.com/>
3. Ultralytics YOLOv8 Documentation. <https://docs.ultralytics.com/>
4. OpenAPI Specification 3.1.0. <https://spec.openapis.org/oas/v3.1.0>
5. Docker Documentation. <https://docs.docker.com/>
6. Pydantic Documentation. <https://docs.pydantic.dev/>
7. OpenCV Documentation. <https://docs.opencv.org/>
8. httpx Documentation. <https://www.python-httpx.org/>
9. Postman Learning Center. <https://learning.postman.com/>
10. Microservices Architecture Pattern. Martin Fowler. <https://martinfowler.com/microservices/>